# Calibration Errors and Trading Opportunities in Kalshi Political Speech Mention Markets

## **Code Availability:** The full analysis code for this paper is available at prediction-market-analysis.


## Abstract

This paper examines the potential for systematic exploitation of persistent calibration errors in Kalshi political speech markets for profit. The dataset includes over 18,757 resolved contracts and is divided into four groups: Trump speech markets, White House press briefings, Zohran Mamdani/NYC mayoral events, and niche political one-offs. I use volume-weighted average price (VWAP) snapshots across eight pre-resolution horizons to estimate temporal calibration curves, then translate those error patterns into implementable trading rules. The strategy layer explicitly assesses fee-adjusted P&L derived from acquiring undervalued outcomes, utilizing Kalshi's precise taker fee schedule.

The results indicate that certain political speech mention subgroups exhibit persistent calibration errors, and descriptive evidence supports a fee-adjusted trading edge.


## 1. Introduction

Prediction markets have evolved from niche instruments used by academics and small trading groups into broadly accessible platforms that aggregate public expectations about real-world events. Market-implied probabilities are now routinely cited alongside polls and expert commentary during national elections, central bank decisions, and geopolitical shocks, owing to their capacity to aggregate and update information in real time. This broader adoption has sharpened a central empirical question: **when do market prices accurately reflect information, and when do they exhibit persistent, tradable errors?**

Prediction markets have also shifted from coarse political predictions (e.g., election winners) toward more granular, high-frequency contracts (e.g., whether a specific word will appear in a scheduled speech). This shift substantially increases the informational yield of market prices. Rather than a small number of sparse, long-term bets, researchers can now observe dense sequences of short-lived contracts that update as new information arrives. Speech mention markets are particularly valuable because their contract language is unambiguous, their

outcomes are rapidly verifiable, and their recurring event formats enable systematic cross-contract comparison.

## 1.1 Longshot Bias — Where It All Began

**Longshot bias** is a well-documented phenomenon in betting and prediction markets in which: low-probability outcomes are systematically **overpriced** and high-probability outcomes are **underpriced** relative to their true likelihood of occurring. Intuitively, traders are often drawn to long-odds contracts because of the possibility of large payoffs, while short-odds contracts offer limited upside and therefore attract less demand. This imbalance in trader preferences pushes longshot prices above fair value and compresses prices for high-probability outcomes below their true probabilities.

In prediction-market practice, this idea maps naturally into a community heuristic often summarized as **"nothing ever happens."**

- **Core intuition:** Unless strong new information arrives, traders tend to default to base-rate continuation.
- **Trading behavior:** Market participants systematically prefer high-probability **"status quo"** contracts rather than headline-driven tail bets.
- **Market culture:** The recurring **"Nothing Ever Happens"** market series on Polymarket — along with surrounding commentary — shows how this heuristic has evolved into an explicit, tradable strategy framework rather than merely a meme.

Viewed through the lens of longshot bias, this heuristic has a clear interpretation: if traders systematically overpay for tail outcomes, the opposite side of those trades should offer positive expected value. In other words, a simple structural strategy emerges — **short longshots and buy favorites**.

We begin by examining the relationship between **win rates and implied probabilities** across all Kalshi markets.

## 1.2 What Are Speech Mention Contracts?

A **speech mention contract** is a binary option that settles to "Yes" or "No" depending on whether a predefined term appears during a specified event window. In economic terms, traders are continuously pricing the probability that speaker $X$ will mention term $Y$ during a particular appearance, conditional on the information available at that moment.

Relative to broader political contracts, this design offers unusually clean empirical structure:

- **Resolution rules are transparent** — a word either appears in a speech or it does not
- **Maturities are short** — markets open hours or days before a speech and close immediately after
- **Event formats recur frequently** — Trump press conferences, White House briefings, and mayoral announcements happen repeatedly, enabling cross-contract comparison

Together, these features create a high-frequency laboratory for measuring calibration error and testing whether those errors remain tradable after fees.

## 1.3 Calibration vs. Trading Profitability

Calibration is an essential characteristic of forecasts; it assesses the alignment of quoted probabilities with actual frequencies across numerous events. Trading profitability, on the other hand, is a different economic issue because even a pricing mistake that can be statistically detected may not be worth anything once transaction costs and fees are taken into account. This paper first establishes whether speech mention market prices are systematically miscalibrated relative to outcomes, and only then asks whether the magnitude of those deviations is large enough to support a net profitable trading strategy after fees.

---

## 1.4 Key Terms & Definitions

| Term | Definition |
|---|---|
| **VWAP** | Volume-Weighted Average Price — the average price of a contract weighted by trade volume at each time point. More representative than simple price because it accounts for how much was traded at each price. $VWAP = \frac{\sum(\text{price} \times \text{volume})}{\sum \text{volume}}$ |
| **Calibration** | A market is *well-calibrated* if contracts priced at X% resolve YES approximately X% of the time. A calibration curve plots implied probability (x-axis) vs. actual resolution rate (y-axis). Perfect calibration = a 45° diagonal. |
| **Miscalibration / Calibration Error** | The gap between implied probability and actual resolution rate. Positive gap = market underprices YES outcomes (favorites underpriced). Negative gap = overpriced. |
| **Longshot Bias** | The empirical pattern where low-probability contracts (longshots) are overpriced and high-probability contracts (favorites) are underpriced. |
| **Pre-resolution horizon** | The time window before a market closes at which we measure the VWAP. E.g., "6h before" means the VWAP computed using all trades up to 6 hours before close. |
| **Taker fee** | Fee paid when you place a market order (immediately fills against existing liquidity). On Kalshi: $\text{Fee}(p) = \left\lceil 0.07 \cdot \frac{p(100-p)}{100} \right\rceil \text{ cents.}$ Where $p = \frac{price_in_cents}{100}$. |
| **Maker order** | A limit order that adds liquidity rather than consuming it. On Kalshi, makers pay **zero fees**, but orders may not fully fill. |
| **EV (Expected Value)** | Expected profit per dollar invested. Positive EV = profitable edge on average. |
| **MAD** | Mean Absolute Deviation — average magnitude of calibration error across price buckets. Lower MAD = better calibrated. $MAD_t = \frac{1}{\lvert\mathcal{P}_t\rvert} \sum_{p \in \mathcal{P}_t} \lvert \text{Win Rate}_t(p) - p \rvert$ |
| **Subgroup** | One of four market categories analyzed: (1) Trump mention markets, (2) White House press briefings, (3) Mamdani/NYC mayoral, (4) Niche one-offs. |

# Prediction Market Analysis: From Longshot Bias to an Exploitable Edge

This analysis traces a research project that began with a simple question: **do prediction markets systematically misprice low-probability and high-probability events?**

The affirmative answer led to a systematic calibration analysis across multiple market categories, ultimately revealing a specific, persistent, and exploitable mispricing in **political speech mention markets on Kalshi**.

## Section 0: Setup

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

plt.style.use("seaborn-v0_8-whitegrid")
plt.rcParams["figure.dpi"] = 120
plt.rcParams["font.size"] = 11

# Color palette
C_BLUE = "#2563eb"
C_GREEN = "#16a34a"
C_RED = "#dc2626"
C_GRAY = "#6b7280"
C_LIGHTBLUE = "#93c5fd"
C_LIGHTRED = "#fca5a5"

strat_colors = [C_BLUE, C_GREEN, "#8b5cf6"]

OUTPUT = Path("output")
SUBGROUPS = OUTPUT / "speech_mention_subgroups"
```

---

## Section 1: Visualizing the Longshot Bias

This section evaluates **market calibration**: when a contract trades at a given price, does that price accurately reflect the contract's empirical probability of resolving in that direction? Put differently, if contracts trade at 70 cents, do positions taken at that price win about 70% of the time?

Calibration is the most direct test of whether market prices can be interpreted as probabilities. Under perfect calibration, contracts priced at $p$ cents should resolve successfully approximately $p\%$ of the time.

## Methodology

The calculation is implemented in a single SQL query built around two common table expressions.

### 1. `resolved_markets`

The first CTE restricts the sample to markets that are **finalized** and have an unambiguous binary outcome, with result equal to either `yes` or `no`. Markets that are unresolved or otherwise ambiguous are excluded, since calibration can only be evaluated when the true outcome is known.

### 2. `all_positions`

The second CTE reconstructs **both sides of every trade**, so that calibration is measured across all positions actually taken in the market.

For the **taker side**, the script records:

- the transaction price actually paid for the position
- whether that position ultimately won, coded as 1 if the chosen side matched the market outcome and 0 otherwise

For the **maker side**, the script records the opposing position from the same trade:

- the corresponding opposite price
- whether that opposing position ultimately won

This construction is important because every executed trade has two economically meaningful sides. If a taker buys YES at 70 cents, then the counterparty is effectively taking the opposite side at 30 cents. The `UNION ALL` combines these two perspectives, so each trade contributes **two observations**: one for the taker and one for the maker.

### 3. Aggregation by price

The final step groups all reconstructed positions by contract price and computes:

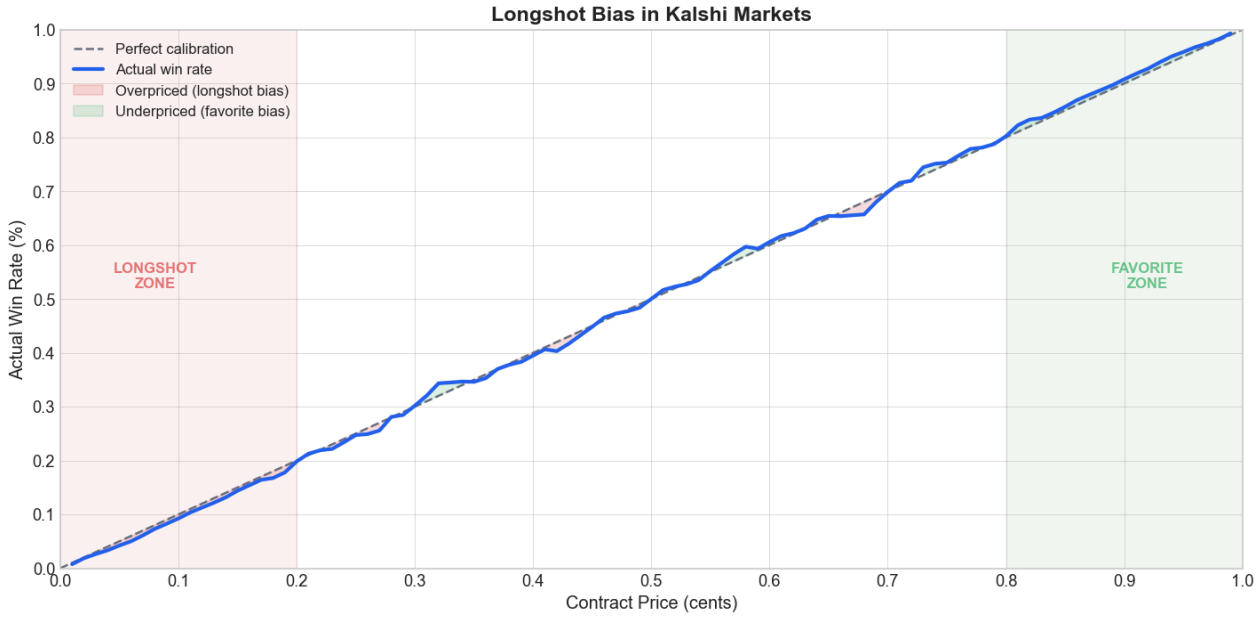
- **total positions**: the number of positions taken at that price
- **wins**: the number of those positions that ultimately resolved successfully
- **win rate**: the fraction of winning positions at that price

Formally, for each price $p$,

$$\text{Win Rate}(p) = \frac{\text{Number of winning positions at price } p}{\text{Total number of positions at price } p}.$$

This gives the empirical resolution frequency for positions entered at each price level.

```
# Load the win rate by price data
df_wr = pd.read_csv(OUTPUT / "win_rate_by_price.csv")

# Convert to implied probability (0-1 scale)
df_wr["implied_prob"] = df_wr["price"] / 100
df_wr["actual_win_rate"] = df_wr["win_rate"] / 100

# Filter to 1-99 range for cleaner plot
mask = (df_wr["price"] >= 1) & (df_wr["price"] <= 99)
df_plot = df_wr[mask].copy()

fig, ax = plt.subplots(figsize=(12, 6))

# Perfect calibration line
ax.plot([0, 1], [0, 1], "--", color=C_GRAY, linewidth=1.5, label="Perfect calibr

# Actual win rate
ax.plot(df_plot["implied_prob"], df_plot["actual_win_rate"], color=C_BLUE, linew

# Shade the gap
ax.fill_between(
    df_plot["implied_prob"], df_plot["actual_win_rate"], df_plot["implied_prob"]
    where=df_plot["actual_win_rate"] < df_plot["implied_prob"],
    alpha=0.15, color=C_RED, label="Overpriced (longshot bias)"
)
ax.fill_between(
    df_plot["implied_prob"], df_plot["actual_win_rate"], df_plot["implied_prob"]
    where=df_plot["actual_win_rate"] > df_plot["implied_prob"],
    alpha=0.15, color=C_GREEN, label="Underpriced (favorite bias)"
)

# Mark zones
ax.axvspan(0, 0.20, alpha=0.06, color=C_RED)
ax.axvspan(0.80, 1.0, alpha=0.06, color=C_GREEN)
ax.text(0.08, 0.52, "LONGSHOT\nZONE", fontsize=10, fontweight="bold", color=C_RE
ax.text(0.92, 0.52, "FAVORITE\nZONE", fontsize=10, fontweight="bold", color=C_GR

# Set x and y axis intervals to 10 cents (0.1)
ax.set_xticks(np.arange(0, 1.1, 0.1))
ax.set_yticks(np.arange(0, 1.1, 0.1))

ax.grid(True, linestyle='-', alpha=0.3, color='gray', linewidth=0.5)

ax.set_xlabel("Contract Price (cents)", fontsize=12)
ax.set_ylabel("Actual Win Rate (%)", fontsize=12)
ax.set_title("Longshot Bias in Kalshi Markets", fontsize=14, fontweight="bold")
ax.legend(loc="upper left", fontsize=10, facecolor='white', edgecolor='gray', fr
ax.set_xlim(0, 1)
ax.set_ylim(0, 1)

plt.tight_layout()
plt.show()
```




## Interpretation

The results are plotted as **actual win rate versus contract price**, with a 45-degree reference line representing perfect calibration:

$$\text{Actual Win Rate} = \text{Price}.$$

If the market is perfectly calibrated, observations should lie on this diagonal, where contracts priced at X% resolve YES approximately X% of the time. For example, positions taken at 70 cents should win about 70% of the time.

- Points **above** the diagonal indicate that contracts at that price win more often than the price implies, meaning they were **underpriced**
- Points **below** the diagonal indicate that contracts at that price win less often than the price implies, meaning they were **overpriced**

This visualization therefore shows whether prediction-market prices behave like well-calibrated probabilities, or whether some price regions systematically overstate or understate the true likelihood of resolution.

## Summary

The red-shaded region at the left confirms classic longshot bias: events priced below ~20% win less often than their price implies. The green-shaded region at the right shows the mirror effect — favorites are underpriced, winning more often than the market suggests.

This is not a small effect. At 10¢, the actual win rate is ~9.2% vs. the 10% implied. But at 5¢, the actual win rate is ~4.2% vs. 5% implied, and the gap widens further at the extremes. On the favorite side, events priced at 90¢ win ~90.8% of the time close to calibrated, but consistently above the line.

## Section 2: Maker Win Rate by Direction

This section tests whether the **longshot bias is visible on the maker side of the market**. The goal is to examine whether makers systematically win more or less often than contract prices imply, depending on whether they are buying **YES** or **NO**.

### Methodology

We first restrict the sample to contracts that are finalized and have a clear binary outcome, with `result` equal to either `yes` or `no`. Unresolved or ambiguous contracts are excluded.

Because the trade data records the **taker's** side, the maker's side is inferred mechanically from the opposite side of the trade:

- if `taker_side = no`, then the maker bought **YES** at `yes_price`
- if `taker_side = yes`, then the maker bought **NO** at `no_price`

For each executed trade, we then define whether the maker ultimately **won** the position:

- a maker who bought **YES** wins if the market resolves `yes`
- a maker who bought **NO** wins if the market resolves `no`

This creates a binary outcome variable equal to 1 if the maker's side matches the final resolution and 0 otherwise.

For each price point and maker direction, the script computes the **contract-weighted maker win rate**:

$$\text{Win Rate}(d, p) = \frac{\sum_{i \in \mathcal{T}_{d,p}} \text{won}_i \cdot \text{contracts}_i}{\sum_{i \in \mathcal{T}_{d,p}} \text{contracts}_i}$$

where:

- $d \in \{\text{YES}, \text{NO}\}$ is the maker's side,
- $p$ is the trade price in cents,
- $\mathcal{T}_{d,p}$ is the set of trades with maker side $d$ executed at price $p$,
- $\text{won}_i \in \{0, 1\}$ indicates whether trade $i$ resolved in favor of the maker,
- and $\text{contracts}_i$ is the number of contracts traded.

This is therefore a **contract-weighted realized win frequency**. Large trades receive proportionally greater weight than small trades.

To evaluate whether makers are trading at favorable prices, the analysis compares this realized win rate to the contract's **implied probability**, given by:

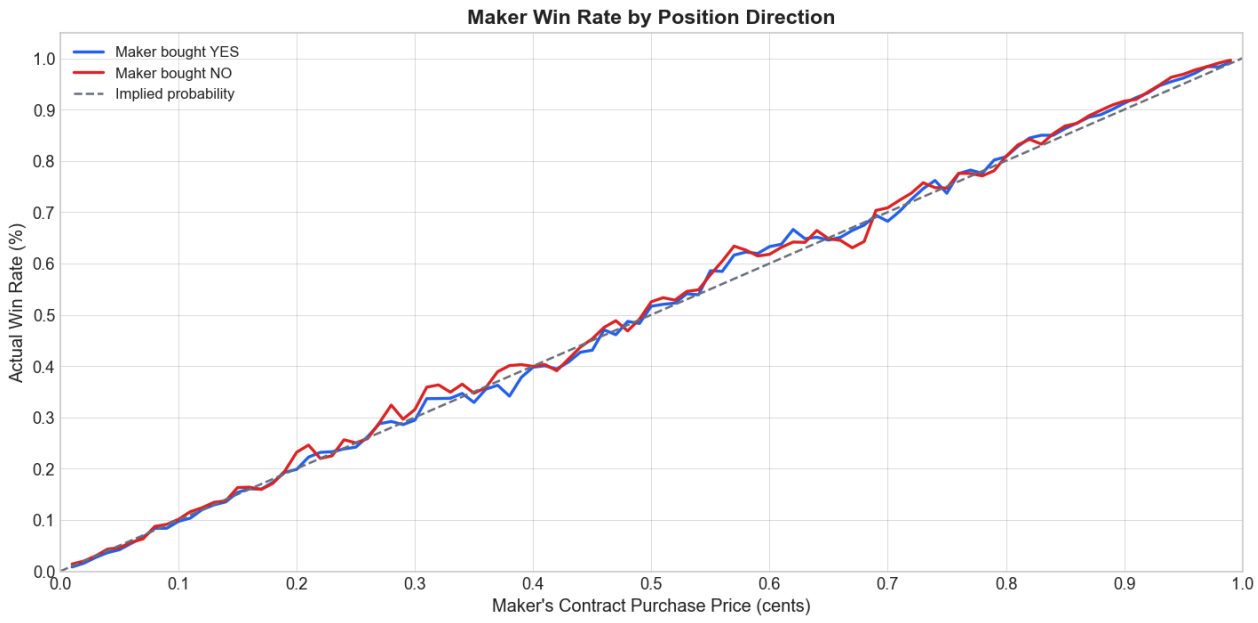
$$\text{Implied Probability}(p) = \frac{p}{100}$$

and defines **mispricing** as

$$\text{Mispricing}(d, p) = \text{Win Rate}(d, p) - \frac{p}{100}.$$

A positive value means that makers buying in direction $d$ at price $p$ win more often than the price implies; a negative value means they win less often than the price implies.

```
# Load maker win rate by direction
df_maker = pd.read_csv(OUTPUT / "maker_win_rate_by_direction.csv")

fig, ax = plt.subplots(figsize=(12, 6))

ax.plot(df_maker["implied_prob"], df_maker["yes_win_rate"], color=C_BLUE, linewi
ax.plot(df_maker["implied_prob"], df_maker["no_win_rate"], color=C_RED, linewidt
ax.plot([0, 1], [0, 1], color=C_GRAY, linestyle="--", linewidth=1.5, label="Impl

# Set x and y axis intervals to 10 cents (0.1)
ax.set_xticks(np.arange(0, 1.1, 0.1))
ax.set_yticks(np.arange(0, 1.1, 0.1))

ax.grid(True, linestyle='-', alpha=0.3, color='gray', linewidth=0.5)

ax.set_xlabel("Maker's Contract Purchase Price (cents)", fontsize=12)
ax.set_ylabel("Actual Win Rate (%)", fontsize=12)
ax.set_title("Maker Win Rate by Position Direction", fontsize=14, fontweight="bo
ax.legend(loc="upper left", fontsize=10, facecolor='white', edgecolor='gray', fr
ax.set_xlim(0, 1)
ax.set_ylim(0, 1.05)

plt.tight_layout()
plt.show()
```



## Interpretation

This is the key object of interest for longshot bias. If low-probability YES contracts are overpriced, then makers who buy **YES** at low prices should underperform relative to the implied probability, while makers who buy **NO** against those same contracts should outperform. In the plot, this should appear as:

- the **YES-maker** win-rate curve falling below the implied-probability line at low prices, and
- the **NO-maker** win-rate curve lying above the implied-probability line in that same region.

## Summary

A pattern like this is exactly what one would expect if longshot claims are systematically overpriced. The section therefore uses maker-side realized win rates by direction and price as evidence that the **directional footprint of longshot bias extends to passive liquidity providers as well**.

The figure above shows that YES makers who provide liquidity at low prices are consistently below break-even. They are the ones *buying* overpriced longshots. NO makers across most of the price range hover near or above break-even. The structural bias favors selling longshots (providing NO liquidity at low prices) and buying favorites.

---

# Section 3: Calibration in NBA Game Contracts

This section examines NBA game contracts as a calibration benchmark. These contracts are among the most liquid on Kalshi, attracting sophisticated bettors, offering unambiguous outcomes, and incorporating extensive public information — conditions under which one would expect strong calibration.

The central question is whether NBA game prediction contracts are well-calibrated before tip-off: if the market assigns a team a 70% chance of winning two hours before tip-off, does that team win approximately 70% of the time?

This analysis excludes live or in-game trading and focuses exclusively on the **pre-game forecast** embedded in market prices, evaluating how well market-implied probabilities summarize available information before the game begins.

## Methodology

The script constructs one pre-game probability estimate per market and then compares those implied probabilities to realized outcomes across probability buckets.

### 1. Market and trade selection

The SQL query begins by identifying all resolved NBA game contracts whose tickers start with `KXNBAGAME-`. Only contracts with a final binary outcome of `yes` or `no` are retained.

It then joins those contracts to their associated trade data and computes a **Volume-Weighted Average Price (VWAP)** for the YES contract using only trades that occurred **at least two hours before market close**. This restriction is important: it removes late-stage or live trading and treats the resulting VWAP as the market's **pre-game implied probability**.

The VWAP is computed as follows:

$$VWAP = \frac{\sum(\text{price} \times \text{volume})}{\sum \text{volume}}$$

The output of this step is one row per market containing:

- the market identifier
- the pre-game YES probability
- the final result
- total trading volume

### 2. Probability normalization

The pre-game VWAP is quoted in cents, so it is divided by 100 to place all probabilities on the standard 0-to-1 scale.

The script also parses the team name from the market ticker, though the central object of interest for calibration is the normalized pre-game probability itself.

### 3. Overall directional accuracy

Before constructing the calibration curve, the script also computes a simple directional accuracy measure. It converts each market probability into a binary forecast:

- if `pre_game_prob > 0.5`, predict **YES**
- if `pre_game_prob ≤ 0.5`, predict **NO**

A forecast is counted as correct if this predicted direction matches the realized outcome. Overall accuracy is then the mean of this correctness indicator across all contracts.

This statistic is useful descriptively, but it is not the main focus of the section. A market can have high directional accuracy and still be poorly calibrated in probability terms. The main object of interest here is calibration.

## Building the Calibration Curve

To evaluate calibration, the script groups contracts into probability buckets and compares the market's implied probability to the empirical frequency of YES outcomes within each bucket.

### 4. Probability buckets

The script divides the probability range from 0 to 1 into 10 equal-width bins:

- 0–10%
- 10–20%
- 20–30%
- ...
- 90–100%

Each market is assigned to one of these buckets based on its pre-game implied probability.

### 5. Empirical resolution frequency

Within each bucket, the script computes:

- **total**: the number of contracts in the bucket
- **actual yes count**: the number of contracts that ultimately resolved YES
- **actual yes %**: the fraction of contracts in that bucket that resolved YES

Formally, for bucket (b),

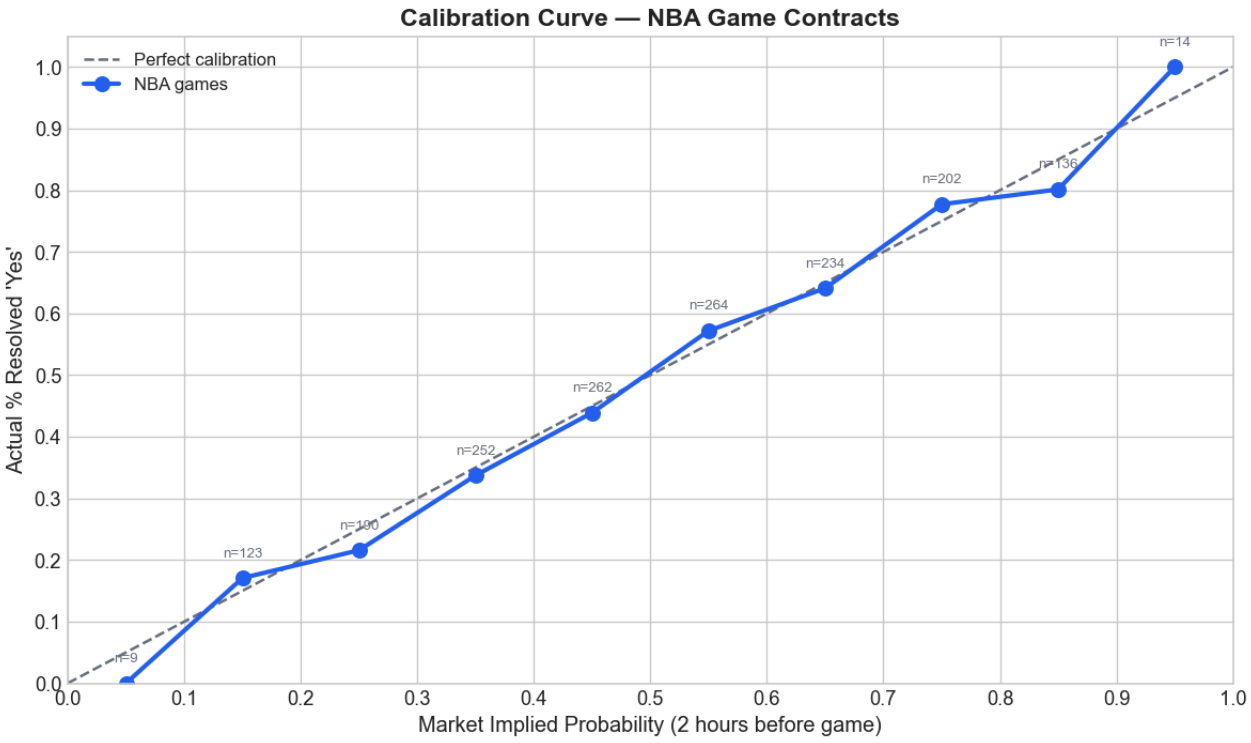
$$\text{Actual YES Rate}(b) = \frac{\text{Number of contracts in bucket } b \text{ that resolved YES}}{\text{Total number of contracts in bucket } b}.$$

This produces the empirical win frequency associated with each range of market-implied probabilities.

The calibration logic is simple: if contracts are well calibrated, then contracts priced around 70% should resolve YES about 70% of the time, contracts priced around 30% should resolve YES about 30% of the time, and so on.

```
# Load NBA calibration data
df_nba = pd.read_csv(OUTPUT / "nba_game_accuracy.csv")

# Parse bucket midpoints
df_nba["bucket_mid"] = df_nba["prob_bucket"].apply(
    lambda x: (int(x.split("-")[0].replace("%", "")) + int(x.split("-")[1].repla
)

fig, ax = plt.subplots(figsize=(10, 6))

# Perfect calibration
ax.plot([0, 1], [0, 1], "--", color=C_GRAY, linewidth=1.5, label="Perfect calibr

# NBA calibration curve
ax.plot(df_nba["bucket_mid"], df_nba["actual_yes_pct"], "o-", color=C_BLUE, line

# Sample size annotations
for _, row in df_nba.iterrows():
    ax.annotate(
        f'n={int(row["total"])}',
```

```
        (row["bucket_mid"], row["actual_yes_pct"]),
        textcoords="offset points", xytext=(0, 12),
        fontsize=8, color=C_GRAY, ha="center"
    )

# Set x and y axis intervals to 10 cents (0.1)
ax.set_xticks(np.arange(0, 1.1, 0.1))
ax.set_yticks(np.arange(0, 1.1, 0.1))

ax.set_xlabel("Market Implied Probability (2 hours before game)", fontsize=12)
ax.set_ylabel("Actual % Resolved 'Yes'", fontsize=12)
ax.set_title("Calibration Curve — NBA Game Contracts", fontsize=14, fontweight="
ax.legend(fontsize=10)
ax.set_xlim(0, 1)
ax.set_ylim(0, 1.05)

plt.tight_layout()
plt.show()
```




## Interpretation

The figure plots the **actual YES rate** for each probability bucket against a benchmark representing **perfect calibration**.

- The **bars** show the observed share of contracts in each bucket that resolved YES
- The **dashed reference line** shows the ideal calibration target, given by the midpoint of each bucket

Under perfect calibration, the empirical YES rate in each bucket should be close to that bucket's midpoint. For example, contracts in the 60–70% bucket should resolve YES at

roughly 65%.

- Bars **above** the benchmark indicate that contracts in that bucket resolved YES more often than implied, meaning the contracts were **underpriced**
- Bars **below** the benchmark indicate that contracts resolved YES less often than implied, meaning the contracts were **overpriced**

This provides a direct visual test of whether NBA game contracts behave like well-calibrated probabilistic forecasts before the game starts.

## Summary

As shown above, **NBA contracts are reasonably well calibrated**, especially in the 20-80% range. The calibration curve hugs the diagonal closely. Small sample sizes at the tails (n=9 for 0-10%, n=14 for 90-100%) make those buckets noisy, but the mid-range is solid — the expected outcome for a deep, well-traded market.

This sets our benchmark: **a well-functioning market looks like this.** Deviations from this pattern in other categories signal potential mispricings.

---

# Section 4: Calibration Deviation Over Time

This section tracks **how Kalshi's calibration evolves over time**, examining not just calibration at a single point but whether market prices become more or less accurate as the platform matures.

The analysis aggregates all resolved Kalshi contracts by week and probability bucket, then compares implied and realized frequencies. The resulting time series reveals whether the platform's pricing accuracy improves over time. It establishes a baseline for comparison with the specific market categories examined in subsequent sections.

In a perfectly calibrated market, the odds match the actual frequencies: contracts priced at 70 cents should win about 70% of the time, contracts priced at 30 cents should win about 30% of the time, and so on. The key metric in this section is therefore **calibration deviation** — the difference between implied win rates and actual win rates.

By computing this deviation cumulatively week by week, the script produces a time series of how closely market prices align with realized outcomes over the sample period.

## Methodology

The calculation proceeds in two stages: first, the script reconstructs all trade positions from resolved contracts; second, it recomputes calibration cumulatively at weekly intervals.

### 1. Constructing trade positions

The SQL query joins historical trades to contracts that have already resolved with a clear binary outcome of `yes` or `no`. Only resolved contracts are included, since calibration can only be evaluated once the true outcome is known.

For each trade, the script reconstructs **both economic sides** of the transaction:

- the **buyer / taker side**, recorded at the price actually paid for that position
- the **counterparty side**, recorded at the complementary price on the opposite outcome

This is important because every trade contains two positions with opposite directional exposure. If one participant buys YES at 60 cents, the other side is effectively taking the complementary NO position at 40 cents. The calibration analysis includes both.

Trades with zero prices are excluded, and the final sample is restricted to prices from **1 to 99 cents**, omitting the degenerate boundary cases at 0 and 100.

### 2. Weekly cumulative calibration snapshots

The script then evaluates calibration at a sequence of weekly endpoints running from the earliest trade date to the latest.

At each weekly boundary, it selects **all trade positions observed up to that date**, not just the trades from that week alone. In other words, the analysis is **cumulative**:

$$\mathcal{D}_t = \{\text{all positions with trade time } \leq t\}.$$

This means each point in the time series summarizes the full calibration record available as of that week.

Within each cumulative sample, the script groups positions by price and computes:

- **total**: the number of positions taken at that price
- **wins**: the number of those positions that ultimately resolved successfully
- **win rate**: the empirical success frequency at that price

For each price $p$, the empirical win rate is

$$\text{Win Rate}_t(p) = 100 \times \frac{\text{Wins up to time } t \text{ at price } p}{\text{Total positions up to time } t \text{ at price } p}.$$

To avoid unstable early estimates, the script skips any weekly snapshot with fewer than 1,000 cumulative trade positions.

## Calibration Deviation Metric

The central statistic is the **mean absolute calibration deviation** across price points.

For each price $p \in \{1, 2, \ldots, 99\}$, the script computes the absolute gap between the realized win rate and the price itself:

$$\text{Deviation}_t(p) = |\text{Win Rate}_t(p) - p| \,.$$

For example:

- if contracts priced at 70 cents have won 65% of the time, the deviation at that price is 5 percentage points
- if contracts priced at 30 cents have won 30% of the time, the deviation is 0, indicating perfect calibration at that price

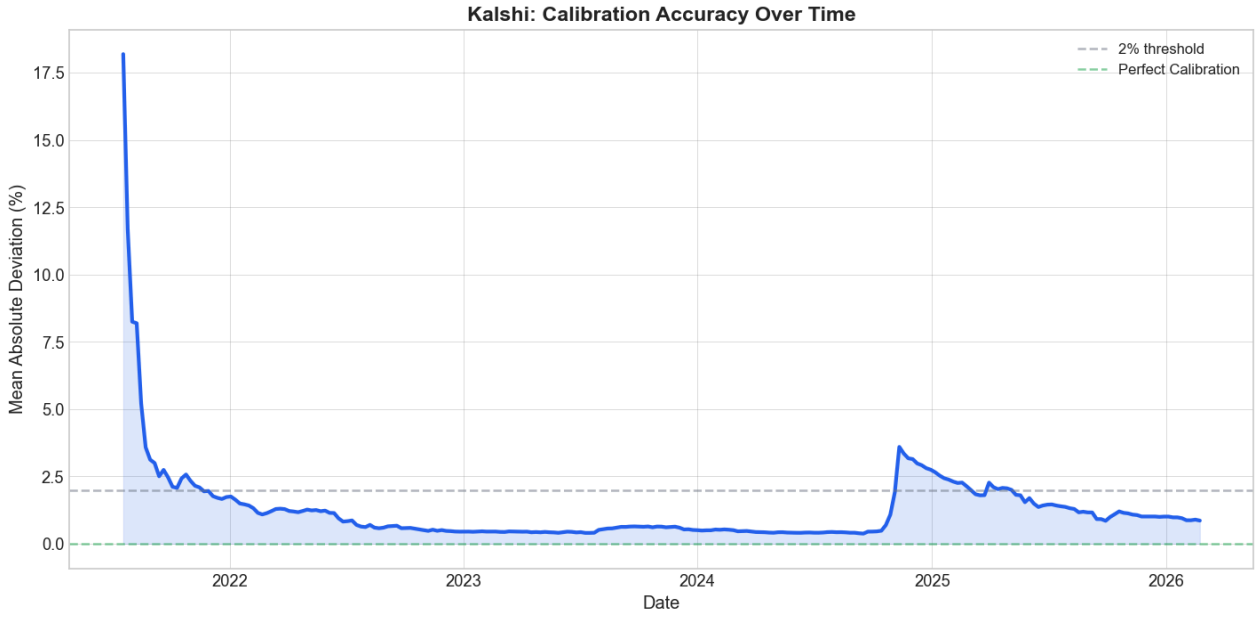
The script then averages these absolute deviations across price points to obtain a single summary statistic for each week:

$$\text{Mean Absolute Deviation}_t = \frac{1}{|\mathcal{P}_t|} \sum_{p \in \mathcal{P}_t} |\text{Win Rate}_t(p) - p| \,,$$

where $\mathcal{P}_t$ is the set of price levels represented in the cumulative sample at time $t$.

This value is measured in **percentage points**. Smaller values indicate that market prices are closer to realized frequencies and therefore better calibrated.

```
# Generate calibration deviation time series from CSV
df_cal = pd.read_csv(OUTPUT / "kalshi_calibration_deviation_over_time.csv")
df_cal["date"] = pd.to_datetime(df_cal["date"])

fig, ax = plt.subplots(figsize=(12, 6))

# Removed marker="o" and markersize=4 to make it a continuous line without dots
ax.plot(df_cal["date"], df_cal["mean_absolute_deviation"], color=C_BLUE, linewid
ax.fill_between(df_cal["date"], df_cal["mean_absolute_deviation"], alpha=0.15, c

ax.grid(True, linestyle='-', alpha=0.3, color='gray', linewidth=0.5)

ax.axhline(y=2, color=C_GRAY, linestyle="--", alpha=0.5, label="2% threshold")
ax.axhline(y=0, color=C_GREEN, linestyle="--", alpha=0.5, label="Perfect Calibra
ax.set_xlabel("Date", fontsize=12)
ax.set_ylabel("Mean Absolute Deviation (%)", fontsize=12)
ax.set_title("Kalshi: Calibration Accuracy Over Time", fontsize=14, fontweight="
ax.legend(fontsize=10)

plt.tight_layout()
plt.show()
```




## Interpretation

The output is a time series of weekly cumulative **mean absolute calibration deviation**, shown as an area chart.

- A value of **0** would indicate perfect calibration
- Higher values indicate that market prices deviate more strongly from realized outcomes
- A downward trend over time suggests that the market is becoming **better calibrated**
- An upward trend suggests that miscalibration is worsening

Because the metric is cumulative, the figure should be interpreted as showing the evolution of the market's **overall historical calibration record**, rather than week-to-week noise. It is therefore best understood as a measure of whether Kalshi's pricing becomes more reliable over time as more data accumulates.

## Summary

The overall Kalshi calibration time series shows that the platform's pricing accuracy fluctuates but generally remains within a few percentage points of perfect calibration for the aggregate market. Most weeks, the MAD hovers near or below the 2% threshold, suggesting that Kalshi's liquid markets (dominated by sports and high-profile events) are reasonably efficient in aggregate.

However, this platform-wide average masks substantial heterogeneity across market categories. The relatively low overall MAD is driven by well-calibrated, high-volume categories like NBA games (Section 3). As the next section will show, specific niche categories — particularly political speech mention contracts — exhibit far larger and more persistent deviations. The overall calibration serves as a useful benchmark: any market category consistently above this baseline is a candidate for further investigation.

# Section 5: Calibration Deviation Over Time in Political Mention Contracts

This section focuses exclusively on *political speech mention contracts*, tracking weekly changes in calibration over the sample period. The metric is the same MAD from Section 4, restricted to contracts whose tickers contain "MENTION" — those that settle based on whether a politician references a specific topic during a speech or briefing.

This portion of the calibration analysis focuses on **political speech mention contracts**: binary claims that settle based on whether a specified term, person, or phrase is mentioned during a political event such as a speech, debate, or public appearance. The objective is to determine whether prices in this market segment behave as well-calibrated probabilities and whether calibration improves or deteriorates over time.

Calibration is assessed by juxtaposing **realized win rates** with the probabilities suggested by contract prices, similar to the broader market-level analysis. The difference is that the sample here is only for a very specific type of market design: short-horizon, event-driven political speech mention contracts. These contracts are highly structured and follow recurring formats, making them well suited for testing whether calibration patterns diverge from the platform-wide baseline.

## Sample Construction

The script applies two layers of filtering so that the final sample contains **political speech mention contracts only**.

### 1. Mention-market filter in SQL

The SQL query first restricts attention to contracts whose ticker or event ticker contains the string `MENTION`. This isolates the family of contracts that settle based on whether something is said or referenced during an event window.

### 2. Political-category filter in Python

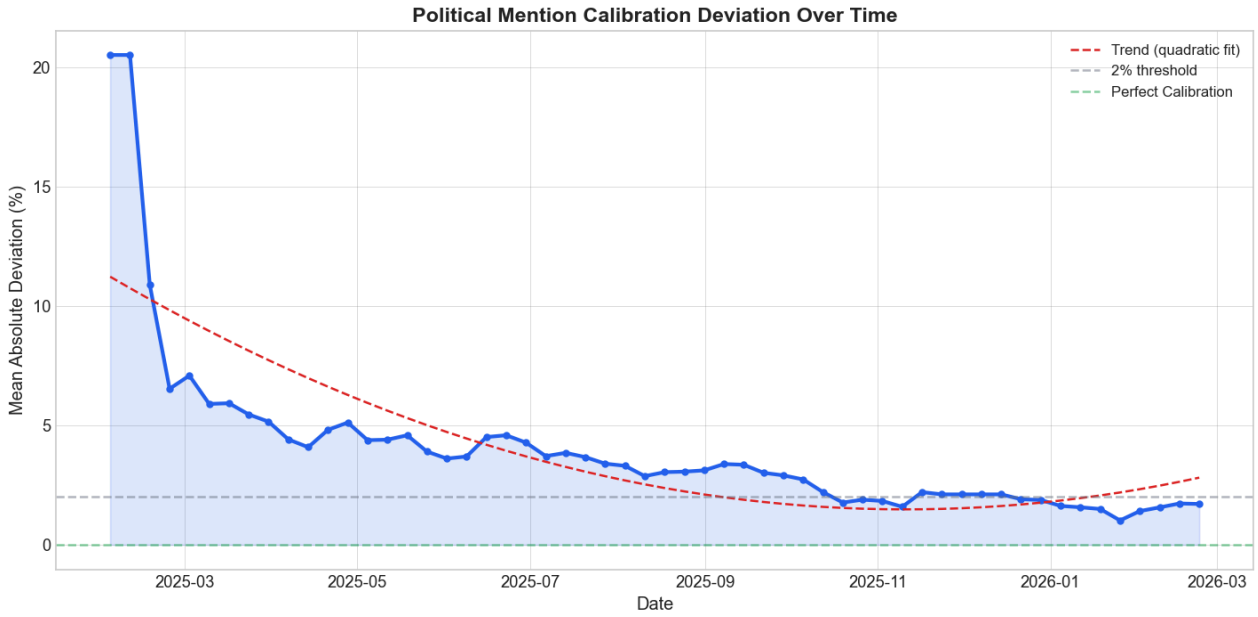
The query also computes a market `category` using the imported `CATEGORY_SQL` classification logic. In Python, each category is then mapped into a broader group, and only observations belonging to the **Politics** group are retained.

This second filter is important because not all mention contracts are political. It removes non-political speech mention contracts — for example, those tied to sports or other domains — and ensures that the analysis focuses specifically on **political speech and event contracts**.

## Methodology

Once the sample is restricted to political speech mention contracts, the calibration calculation follows the same structure as **(Section 4) Calibration Deviation Over Time.**

```
# Generate political calibration deviation time series from CSV
df_cal = pd.read_csv(OUTPUT / "kalshi_political_mention_calibration_deviation_ov
df_cal["date"] = pd.to_datetime(df_cal["date"])

fig, ax = plt.subplots(figsize=(12, 6))

ax.plot(df_cal["date"], df_cal["mean_absolute_deviation"], color=C_BLUE, linewid
ax.fill_between(df_cal["date"], df_cal["mean_absolute_deviation"], alpha=0.15, c

# Trend line
z = np.polyfit(range(len(df_cal)), df_cal["mean_absolute_deviation"], 2)
p = np.poly1d(z)
ax.plot(df_cal["date"], p(range(len(df_cal))), "--", color=C_RED, linewidth=1.5,

ax.grid(True, linestyle='-', alpha=0.3, color='gray', linewidth=0.5)

ax.axhline(y=2, color=C_GRAY, linestyle="--", alpha=0.5, label="2% threshold")
ax.axhline(y=0, color=C_GREEN, linestyle="--", alpha=0.5, label="Perfect Calibra
ax.set_xlabel("Date", fontsize=12)
ax.set_ylabel("Mean Absolute Deviation (%)", fontsize=12)
ax.set_title("Political Mention Calibration Deviation Over Time", fontsize=14, f
ax.legend(fontsize=10)

plt.tight_layout()
plt.show()
```




## Interpretation

The resulting time series shows how calibration in **political speech mention contracts** evolves as more data accumulates.

- A **lower MAD** means prices more closely match true outcome frequencies
- A **higher MAD** means the market is less well calibrated
- A **declining trend** suggests that calibration is improving over time
- A **persistently elevated MAD** suggests that this market type contains stable and recurring mispricing

This section is especially important because political speech mention contracts are precisely the type of high-frequency, text-defined contracts where longshot-style distortions may be most visible. If their calibration deviation remains materially above zero over time, that is evidence that these prices do not fully behave like unbiased probability estimates, even after the market has matured.

## Summary

The time series reveals a striking pattern of initial extreme miscalibration followed by partial convergence. Early political speech mention contracts (February 2025) exhibited MAD values near 20%, meaning prices were systematically off by a fifth of the probability scale. By late 2025, the deviation compressed to roughly 2%, consistent with a market that is learning but has not fully corrected.

The quadratic trend line captures this nonlinear decay well. Crucially, the curve flattens rather than continuing toward zero, suggesting a structural floor to the mispricing. This plateau is consistent with thin liquidity and limited arbitrage capital in niche political contracts. The cumulative series suggests partial convergence but not full disappearance of miscalibration.

```
# Analyze the deviation data
print("=== Political Mention Calibration Deviation Over Time ===")
print(f"Mean absolute deviation (overall): {df_cal['mean_absolute_deviation'].me
print(f"Early period (first 8 weeks):      {df_cal['mean_absolute_deviation'].he
print(f"Recent period (last 8 weeks):       {df_cal['mean_absolute_deviation'].t

# Simple trend
slope = np.polyfit(range(len(df_cal)), df_cal["mean_absolute_deviation"], 1)[0]
print(f"\nTrend: {slope:.3f} percentage points/week ({'improving' if slope < 0 e
```

| Metric | Value |
|---|---|
| **Mean Absolute Deviation (Overall)** | **4.03 percentage points** |
| **Early Period (First 8 Weeks)** | **10.35%** |
| **Recent Period (Last 8 Weeks)** | **1.52%** |

**Trend:** −0.153 percentage points per week *(improving)*

Early contracts exhibit large pricing deviations (≈10.35%), suggesting limited trader experience or thinner liquidity. In contrast, the most recent contracts show much tighter

calibration (≈1.52%), consistent with **market learning and improved price discovery** as traders accumulate experience with the contract format.

---

# Section 6: Calibration in Political Speech Mention Contracts

**Political speech mention contracts** are contracts like *"Will Trump mention immigration in his State of the Union address?"* or *"Will the White House press briefing mention Ukraine?"*

This section constructs a calibration curve to evaluate the accuracy of Kalshi's implied probabilities. Contracts are grouped into 10-percentage-point probability buckets based on their implied price. For example, all contracts trading between 60 and 70 cents are assigned to the same bucket, and the actual YES resolution rate is computed for each. Under perfect calibration, every point would lie on the 45-degree diagonal. The central question is whether a contract priced at 70% two hours before a speech resolves YES approximately 70% of the time.

There are two calibration curves: one for the NBA games baseline (from Section 3) and one for the political speech mention contracts. The NBA curve serves as a benchmark for well-calibrated pricing on Kalshi. The shaded regions highlight the tails where exploitable divergence is most likely: the low-probability area (where longshot overpricing would appear) and the high-probability area (where favorite underpricing — the principal finding of this paper — is concentrated). Sample sizes ( `n=` ) are annotated at each point to indicate statistical power.

As in the NBA analysis, the focus is on the market's **pre-event forecast** rather than late trading immediately before resolution, testing whether prices in these short-term political contracts function as genuine probabilities.

## Methodology

The script constructs one pre-speech probability estimate per market, filters to political speech mention contracts, and then compares those implied probabilities to realized outcomes across probability buckets.

Once the sample is restricted to political speech mention contracts, the calibration calculation follows the same structure as **(Section 3) Calibration NBA Game contracts.**

### 1. Market and trade selection

The SQL query begins by identifying all resolved mention contracts whose `ticker` or `event_ticker` contains `MENTION`. Only contracts with a final binary outcome of `yes` or `no` are retained.

It then joins those contracts to their associated trade data and computes a **Volume-Weighted Average Price (VWAP)** for the YES contract using only trades that occurred **at least two hours before market close**. This restriction is important: it excludes trading that occurs too close to resolution and treats the resulting VWAP as the market's **pre-speech implied probability**.

The output of this step is one row per market containing:

- the market identifier
- the pre-speech YES probability
- the final result
- total trading volume

### 2. Political-market filtering

After the SQL stage, the script applies an additional Python-side filter to isolate the relevant subset of contracts. Each market is assigned a group using `get_group(category)`, and only contracts where `group == "Politics"` are kept.

This step matters because not all mention contracts are political. The post-query filter ensures that the analysis is focused specifically on **political speech mention contracts**, excluding unrelated mention contracts from other domains.

### 3. Title parsing and market structure

The script also extracts structured information from market titles using regular expressions. In particular, it parses each title to identify:

- the **speaker**
- the **event type**

For example, the title may specify a named politician and a particular appearance, such as a press briefing, debate, or public address. This parsing step is not required for the aggregate calibration curve itself, but it enables later breakdowns by speaker and by event type.

## Breakdowns by Speaker and Event

In addition to the aggregate calibration curve, the script also reports two drill-down views:

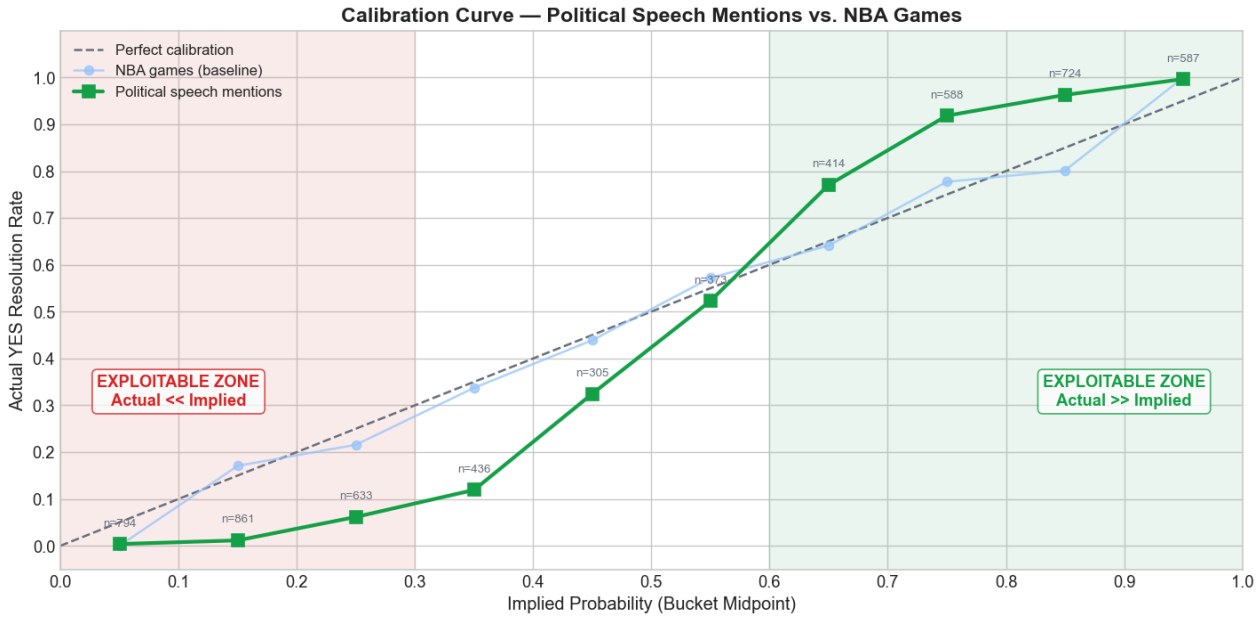
- **by speaker**: accuracy statistics for the most active speakers
- **by event**: accuracy statistics for the most common event types

These summaries help show whether forecast quality varies systematically across political actors or across different kinds of public appearances. Some speakers may be easier for contracts to anticipate than others, and some event formats may generate more predictable language patterns than others.

The script restricts these breakdowns to the top entries by volume, which keeps the comparisons focused on the most active and informative subsets of the data.

```
# Load political speech mention calibration data
df_pol = pd.read_csv(OUTPUT / "political_speech_mention_accuracy.csv")

df_pol["bucket_mid"] = df_pol["prob_bucket"].apply(
    lambda x: (int(x.split("-")[0].replace("%", "")) + int(x.split("-")[1].repla
)

fig, ax = plt.subplots(figsize=(12, 6))

# Perfect calibration
ax.plot([0, 1], [0, 1], "--", color=C_GRAY, linewidth=1.5, label="Perfect calibr

# NBA baseline (lighter)
ax.plot(df_nba["bucket_mid"], df_nba["actual_yes_pct"], "o-", color=C_LIGHTBLUE,
        markersize=6, alpha=0.7, label="NBA games (baseline)")

# Political speech calibration
ax.plot(df_pol["bucket_mid"], df_pol["actual_yes_pct"], "s-", color=C_GREEN, lin
        markersize=8, label="Political speech mentions")

# Highlight the divergence zone
ax.axvspan(0.60, 1.0, alpha=0.08, color=C_GREEN, zorder=1)
ax.axvspan(0.0, 0.30, alpha=0.08, color=C_RED, zorder=1)

ax.annotate(
    "EXPLOITABLE ZONE\nActual >> Implied",
    xy=(0.90, 0.30), fontsize=11, fontweight="bold", color=C_GREEN,
    ha="center",
    bbox=dict(boxstyle="round,pad=0.3", facecolor="white", edgecolor=C_GREEN, al
)

ax.annotate(
    "EXPLOITABLE ZONE\nActual << Implied",
    xy=(0.10, 0.30), fontsize=11, fontweight="bold", color=C_RED,
    ha="center",
    bbox=dict(boxstyle="round,pad=0.3", facecolor="white", edgecolor=C_RED, alph
)

# Sample size annotations for political
for _, row in df_pol.iterrows():
    ax.annotate(
        f'n={int(row["total"])}',
        (row["bucket_mid"], row["actual_yes_pct"]),
        textcoords="offset points", xytext=(0, 12),
        fontsize=8, color=C_GRAY, ha="center"
    )


# Set x and y axis intervals to 10 cents (0.1)
ax.set_xticks(np.arange(0, 1.1, 0.1))
ax.set_yticks(np.arange(0, 1.1, 0.1))

ax.set_xlabel("Implied Probability (Bucket Midpoint)", fontsize=12)
```

```
ax.set_ylabel("Actual YES Resolution Rate", fontsize=12)
ax.set_title("Calibration Curve — Political Speech Mentions vs. NBA Games", font
ax.legend(fontsize=10, loc="upper left")
ax.set_xlim(0, 1)
ax.set_ylim(-0.05, 1.1)

plt.tight_layout()
plt.show()
```




## Interpretation

The figure plots the **actual YES rate** for each probability bucket against a benchmark representing **perfect calibration**.

- The **squares** show the observed share of contracts in each bucket that resolved YES
- The **dashed reference line** shows the ideal calibration target, given by the midpoint of each bucket

Under perfect calibration, the empirical YES rate in each bucket should be close to that bucket's midpoint. For example, contracts in the 60–70% bucket should resolve YES at roughly 65%.

- Bars **above** the benchmark indicate that contracts in that bucket resolved YES more often than implied, meaning the contracts were **underpriced**
- Bars **below** the benchmark indicate that contracts resolved YES less often than implied, meaning the contracts were **overpriced**

This provides a direct visual test of whether political speech mention contracts behave like well-calibrated probabilistic forecasts before the speech occurs.

## Summary

The calibration curve for political speech mention contracts diverges dramatically from the diagonal in the high-probability zone. While the NBA baseline tracks the 45-degree line closely (as expected for a deep, liquid market), speech mention contracts priced above 60¢ systematically resolve YES far more often than their prices imply. The gap widens from +12% at the 60–70% bucket to a peak of +17% at 70–80%, before narrowing at the extreme tail (90–100%).

This pattern is consistent with the classic longshot bias documented in horse racing and sports betting literature: longshot contracts are overpriced while favorites are significantly underpriced. The large sample sizes (hundreds of contracts per bucket) lend statistical significance to these deviations. This is the paper's headline result: a structural, persistent, and economically large mispricing in a specific Kalshi market category.

```
# Quantify the gap in the high-probability zone
print("=== High-Probability Zone Comparison (>60% implied) ===")
print(f"{'Bucket':<12} {'NBA Actual':>12} {'Speech Actual':>14} {'Speech Gap':>1
print("-" * 52)

for _, pol_row in df_pol[df_pol["bucket_mid"] >= 0.6].iterrows():
    nba_match = df_nba[df_nba["prob_bucket"] == pol_row["prob_bucket"]]
    nba_val = nba_match["actual_yes_pct"].values[0] if len(nba_match) > 0 else f
    gap = pol_row["actual_yes_pct"] - pol_row["bucket_mid"]
    print(f"{pol_row['prob_bucket']:<12} {nba_val:>11.1%} {pol_row['actual_yes_p
```

## Speech Mention Contracts vs. Calibration Benchmark

Below we compare the **actual YES resolution frequency** with the **implied probability midpoint** of each bucket.

### NBA Game Contracts

| Probability Bucket | Actual YES Rate | Calibrated Probability | Calibration Gap |
|---|---|---|---|
| 60–70% | **64.1%** | 65% | **−0.9%** |
| 70–80% | **77.7%** | 75% | **+2.7%** |
| 80–90% | **100.0%** | 85% | **+5.0%** |

### Political Speech Mention Contracts

| Probability Bucket | Actual YES Rate | Calibrated Probability | Calibration Gap |
|---|---|---|---|
| 60–70% | **77.1%** | 65% | **+12.1%** |
| 70–80% | **91.8%** | 75% | **+16.8%** |
| 80–90% | **96.3%** | 85% | **+11.3%** |

**This is the core finding.** Political speech mention contracts are massively miscalibrated in the favorite zone:

By comparison, NBA contracts in the same buckets fall within a few percentage points of the diagonal. Speech mention contracts are **dramatically underpricing favorites**.

---

# Section 7: Speech Mention Market Accuracy and Calibration by Subgroup

This section studies whether **political speech mention contracts behave differently across speaker and event types**. Rather than treating all mention contracts as a single pool, the analysis breaks them into recurring subgroups and asks three related questions:

1. **Directional accuracy:** does the market's majority view correctly predict the final outcome?
2. **Calibration:** do quoted probabilities match realized frequencies within each subgroup?
3. **Profitability:** if a trader naively follows extreme market probabilities, are the resulting trades profitable after taker fees?

This subgroup structure matters because speech mention contracts are not homogeneous. A market on whether Donald Trump will mention a phrase may behave very differently from a White House press briefing or a niche one-off political event. Grouping contracts by speaker and event type therefore allows the analysis to test whether calibration quality and tradability vary systematically across market contexts.

## Data Construction

The script begins by loading resolved speech mention contracts and their associated trade histories using DuckDB.

### 1. Market selection

The SQL query restricts the sample to contracts whose ticker or event ticker contains `MENTION` and whose final result is unambiguously `yes` or `no`. After loading, the sample is further restricted to contracts classified in the **Politics** group, ensuring that non-political speech mention contracts are excluded.

The subgroups are assigned using the event ticker prefix: `TRUMPMENTION*` for Trump speeches, `SECPRESSMENTION*` for White House press briefings, `MAMDANIMENTION*` / `NYCMAYORDEBATEMENTION*` for NYC mayoral events, and everything else as niche/one-off.

### 2. Pre-event probability snapshots

For each market, the script computes **Volume-Weighted Average Prices (VWAPs)** for the YES side at multiple horizons before market close:

- 7 days
- 3 days
- 1 day
- 6 hours
- 2 hours
- 1 hour
- 30 minutes
- 10 minutes

Each VWAP is calculated using only trades that occurred **before** the relevant cutoff. For example, the 2-hour VWAP uses only trades placed at least two hours before market close. These snapshots are intended to approximate the market's implied probability at different stages of information arrival leading up to the event.

## Speaker and Event Subgroups

To study heterogeneity across contract types, the script parses market titles using regular expressions to identify the relevant speaker and event context. It then assigns each market to one of four subgroups:

| Subgroup | Definition |
|---|---|
| **Group 1: Trump Markets** | Contracts involving Donald Trump / President Trump |
| **Group 2: Press Briefings** | Contracts involving White House press briefings |
| **Group 3: Mayoral** | Contracts involving Zohran Mamdani or the NYC Mayor's Office |
| **Group 4: Niche / One-offs** | All remaining political speech mention contracts |

This classification allows the analysis to compare large, recurring market types against smaller and less standardized contracts.

---

# Section 7a: Directional Accuracy

The first summary statistic is **overall directional accuracy**, based on the market's 2-hour pre-event probability.

For each contract:

- define `actual_yes` as whether the market ultimately resolved YES
- define `predicted_yes` as whether the 2-hour VWAP exceeds 50 cents
- define `correct` as whether the prediction matches the realized outcome

Formally,

$$\text{Predicted YES}_i = \mathbf{1}(\text{VWAP}_{2h,i} > 0.5)\,,$$

and

$$\text{Accuracy}_g = \frac{1}{N_g} \sum_{i \in g} \mathbf{1}(\text{Predicted YES}_i = \text{Actual YES}_i)\,,$$

where (g) indexes the subgroup.

In short, if the market price 2 hours before close is above 50¢, it's predicting the event will be mentioned. If it's below 50¢, predicting it won't. Accuracy is the percentage of times this naive threshold prediction was right. The horizontal bar chart visualizes these accuracy rates with contract counts annotated, allowing quick assessment of both the magnitude of the edge and the available trading volume.

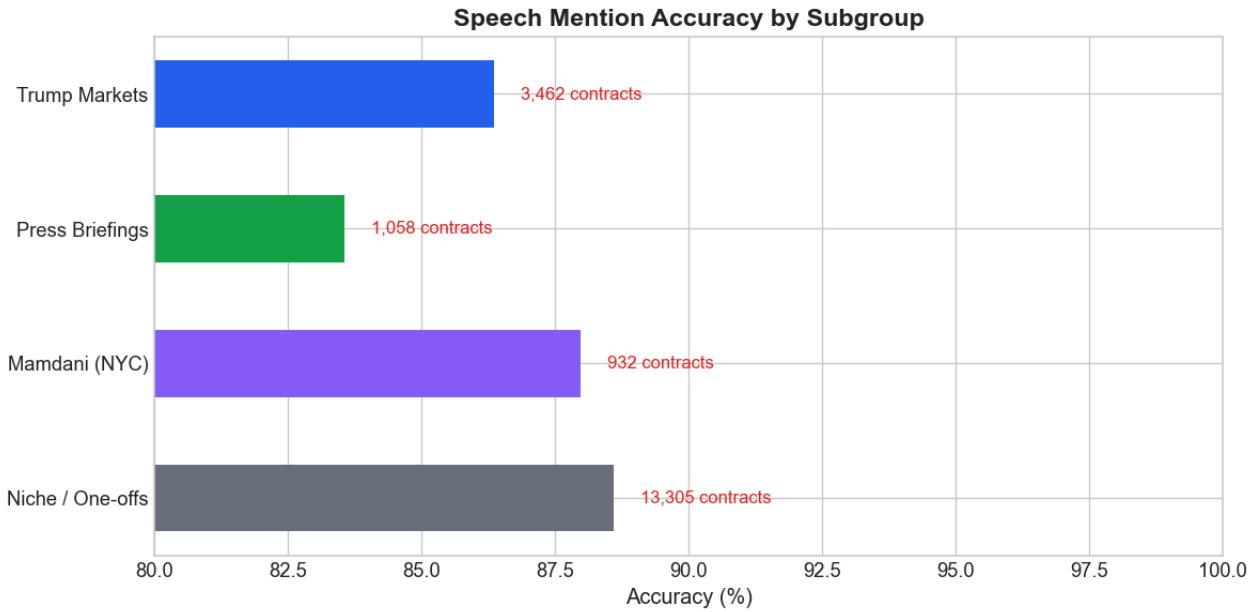
This statistic answers a simple question: **how often was the market directionally correct two hours before resolution?** It is useful as a descriptive benchmark, but it should not be confused with calibration. A subgroup can have high directional accuracy while still being systematically overconfident or underconfident in probability terms.

```
import duckdb
import math

TRADES = "data/kalshi/trades/*.parquet"
MARKETS = "data/kalshi/markets/*.parquet"

con = duckdb.connect()

# Load all MENTION markets with VWAPs at each time window (runs once, reused by
print("Loading mention market data (this takes a few minutes)...")
df_mention = con.execute(f"""
WITH mb AS (
    SELECT ticker, title, result, close_time,
        regexp_extract(event_ticker, '^([A-Z0-9]+)', 1) AS category
    FROM '{MARKETS}'
    WHERE result IN ('yes', 'no')
      AND (ticker LIKE '%MENTION%' OR event_ticker LIKE '%MENTION%')
),
vwaps AS (
    SELECT t.ticker,
        AVG(CASE WHEN t.created_time <= m.close_time - INTERVAL 1 DAY THEN CAST(
        AVG(CASE WHEN t.created_time <= m.close_time - INTERVAL 6 HOUR THEN CAST
        AVG(CASE WHEN t.created_time <= m.close_time - INTERVAL 2 HOUR THEN CAST
        AVG(CASE WHEN t.created_time <= m.close_time - INTERVAL 1 HOUR THEN CAST
        AVG(CASE WHEN t.created_time <= m.close_time - INTERVAL 30 MINUTE THEN C
        AVG(CASE WHEN t.created_time <= m.close_time - INTERVAL 10 MINUTE THEN C
```

```
        SUM(t.count) as total_volume
    FROM '{TRADES}' t
    JOIN mb m ON t.ticker = m.ticker
    GROUP BY t.ticker
)
SELECT mb.ticker, mb.title, mb.result, mb.category, vwaps.*
FROM mb JOIN vwaps ON mb.ticker = vwaps.ticker
WHERE vwaps.vwap_2h IS NOT NULL
""").df()
print(f"Loaded {len(df_mention)} contracts")

# Assign subgroups based on category code
def assign_subgroup(row):
    c = row['category'].upper()
    if 'TRUMPMENTION' in c or 'TRUMPMENTIONB' in c:
        return 'Group 1: Trump Markets'
    elif 'SECPRESSMENTION' in c:
        return 'Group 2: Press Briefings'
    elif 'MAMDANIMENTION' in c or 'NYCMAYORDEBATEMENTION' in c:
        return 'Group 3: Mayoral (Zohran Mamdani)'
    else:
        return 'Group 4: Niche / One-offs'

df_mention['subgroup'] = df_mention.apply(assign_subgroup, axis=1)

# Compute df_sub from df_mention (single source of truth — replaces CSV read)
def kalshi_fee(price_cents):
    return math.ceil(0.07 * price_cents * (100.0 - price_cents) / 100.0)

_rows = []
for g in ['Group 1: Trump Markets', 'Group 2: Press Briefings',
          'Group 3: Mayoral (Zohran Mamdani)', 'Group 4: Niche / One-offs']:
    g_df = df_mention[df_mention['subgroup'] == g].copy()
    price = g_df['vwap_2h']
    actual_yes = g_df['result'] == 'yes'
    predicted_yes = price > 50
    acc = (predicted_yes == actual_yes).mean()

    def _buy_yes_profit(mask):
        sub = g_df[mask]
        if len(sub) == 0:
            return 0, 0.0
        p = sub['vwap_2h']
        yes = (sub['result'] == 'yes').astype(float)
        fee = p.apply(kalshi_fee)
        cost = (p + fee).sum()
        payout = (yes * 100).sum()
        return len(sub), float((payout - cost) / cost) if cost > 0 else 0.0

    # Extremes: buy YES when >70, buy NO when <30
    buy_yes_ext = g_df[price > 70]
    buy_no_ext = g_df[price < 30]
    ext_trades = len(buy_yes_ext) + len(buy_no_ext)
    if ext_trades > 0:
        p_y = buy_yes_ext['vwap_2h']
        cost_y = (p_y + p_y.apply(kalshi_fee)).sum()
        pay_y = ((buy_yes_ext['result'] == 'yes').astype(float) * 100).sum()
```

```
        p_n = buy_no_ext['vwap_2h']
        cost_n = ((100 - p_n) + p_n.apply(kalshi_fee)).sum()
        pay_n = ((buy_no_ext['result'] == 'no').astype(float) * 100).sum()
        ext_profit = float((pay_y + pay_n - cost_y - cost_n) / (cost_y + cost_n)
    else:
        ext_profit = 0.0

    g60_t, g60_p = _buy_yes_profit(price > 60)
    g70_t, g70_p = _buy_yes_profit(price > 70)
    g80_t, g80_p = _buy_yes_profit(price > 80)
    g90_t, g90_p = _buy_yes_profit(price > 90)

    _rows.append({
        'subgroup': g, 'total_mention_contracts': len(g_df), 'accuracy': acc,
        'ext_trades': ext_trades, 'ext_profit': ext_profit,
        'gt60_trades': g60_t, 'gt60_profit': g60_p,
        'gt70_trades': g70_t, 'gt70_profit': g70_p,
        'gt80_trades': g80_t, 'gt80_profit': g80_p,
        'gt90_trades': g90_t, 'gt90_profit': g90_p,
    })

df_sub = pd.DataFrame(_rows)

# Summary table
summary = df_sub[["subgroup", "total_mention_contracts", "accuracy"]].copy()
summary["accuracy_pct"] = (summary["accuracy"] * 100).round(1)
summary = summary.rename(columns={
    "subgroup": "Subgroup",
    "total_mention_contracts": "Contracts",
    "accuracy_pct": "Accuracy (%)"
})

print("=== Speech Mention Subgroup Summary ===")
print(summary[["Subgroup", "Contracts", "Accuracy (%)"]].to_string(index=False))
```

```
# Horizontal bar chart: accuracy by subgroup
fig, ax = plt.subplots(figsize=(10, 5))

colors = [C_BLUE, C_GREEN, "#8b5cf6", C_GRAY]
short_labels = ["Trump Markets", "Press Briefings", "Mamdani (NYC)", "Niche / On

bars = ax.barh(short_labels[::-1], df_sub["accuracy"].values[::-1] * 100, color=

# Add contract count annotations
for bar, n in zip(bars, df_sub["total_mention_contracts"].values[::-1]):
    ax.text(bar.get_width() + 0.5, bar.get_y() + bar.get_height() / 2,
            f"{n:,} contracts", va="center", fontsize=10, color=C_RED)

ax.set_xlabel("Accuracy (%)", fontsize=12)
ax.set_title("Speech Mention Accuracy by Subgroup", fontsize=14, fontweight="bol
ax.set_xlim(80, 100)

plt.tight_layout()
plt.show()
```




**All four subgroups show accuracy well above what market prices imply** (the average implied probability is well below 80% for most contracts). Niche markets are the most accurate at approximately 88.6%, followed by Mamdani and Trump markets at approximately 88% and 86.4%, while press briefings are the least accurate at approximately 83.6%.

The following subsections examine each subgroup in detail.

---

# Section 7a: Trump Contracts

Trump mention contracts are the **largest and most liquid** subgroup (see summary table above for exact counts). These are contracts like *"Will Trump mention [topic] in his next speech/press conference?"*

The mispricing here is substantial: prices frequently trade below 70¢ for events that resolve YES over 90% of the time. This is the most promising category for a systematic strategy.

This deep dive examines Trump mention contracts using two complementary visualizations. *1-cent, 5-cent, and 10-cent calibration charts* bins contracts into narrow 1¢, 5¢ and 10¢ wide price buckets and compares the implied probability to the actual resolution rate within each bin.

## Methodology

Once the sample is restricted to political speech mention contracts, the calibration calculation follows the same structure as **(Section 3) Calibration NBA Game contracts.**

### 1. Bucket construction

For each subgroup and each pre-event horizon, the script bins contracts into probability buckets at three levels of granularity:

- **10-cent buckets**
- **5-cent buckets**
- **1-cent buckets**

Within each bucket, it computes the empirical share of contracts that ultimately resolved YES.

### 2. Actual vs. predicted frequency

For a bucket (b), let (p_b) denote the bucket midpoint and let the contracts in that bucket be those whose pre-event VWAP falls in that range. The realized YES rate is

$$\text{Actual YES Rate}(b) = \frac{\text{Number of contracts in bucket } b \text{ resolving YES}}{\text{Total number of contracts in bucket } b}.$$

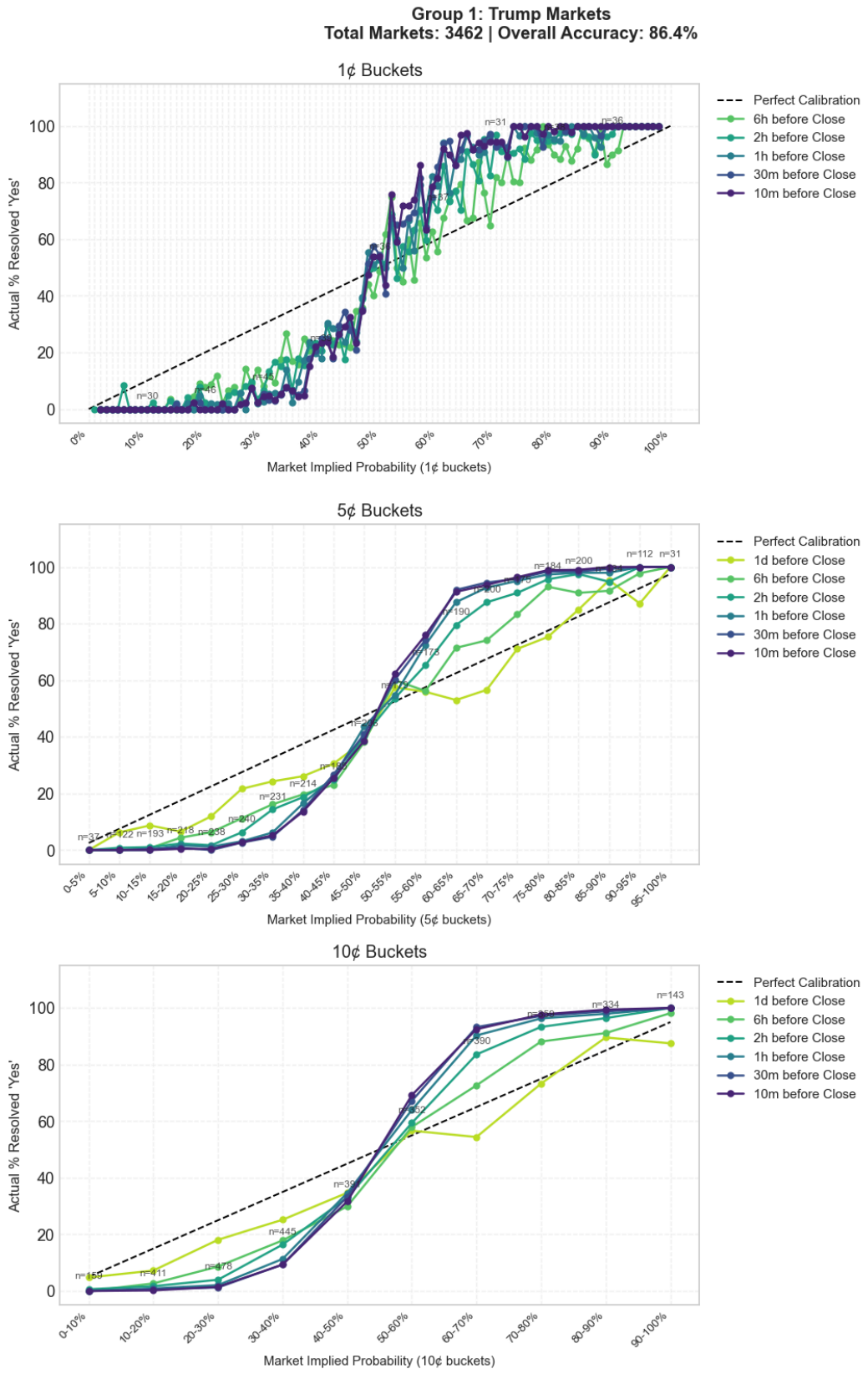
If contracts are well calibrated, this realized YES rate should be close to the bucket midpoint. For example, contracts in a 70–80% bucket should resolve YES about 75% of the time.

```
# Reuse df_sub (computed from df_mention above) for calibration chart titles
sg = df_sub

def plot_calibration(group_df, group_name, total_contracts, accuracy, bin_size=5
    if facet:
        facet_bins = [1, 5, 10]
        # Compact vertical stacking: 3 rows, 1 column, reduced height for PDF fi
        fig, axes = plt.subplots(3, 1, figsize=(10, 14), sharey=True)

        for i, bs in enumerate(facet_bins):
            _plot_calibration_ax(axes[i], group_df, bs)
            axes[i].set_title(f"{bs}¢ Buckets", fontsize=12)
            axes[i].set_ylabel("Actual % Resolved 'Yes'", fontsize=10)
            axes[i].legend(loc="upper left", bbox_to_anchor=(1.01, 1), fontsize=

        fig.suptitle(f"{group_name}\nTotal Markets: {total_contracts} | Overall
                     fontsize=12, fontweight='bold', y=0.95)

        plt.tight_layout(rect=[0, 0, 0.86, 0.96]) # Leave room for legend and su
        plt.show()
        return

    # Single plot (original behavior)
    fig, ax = plt.subplots(figsize=(10, 6))
    _plot_calibration_ax(ax, group_df, bin_size)
    ax.set_ylabel("Actual % Resolved 'Yes'")
    ax.set_title(f"{group_name}\nTotal Markets: {total_contracts} | Overall Accu
    ax.legend(loc="upper left", bbox_to_anchor=(1.01, 1))
    plt.tight_layout()
    plt.show()
```

```python
def _plot_calibration_ax(ax, group_df, bin_size):
    """Draw calibration lines on a single Axes for the given bin_size."""
    if bin_size == 1:
        bins = np.arange(-0.5, 101.5, 1)
        n_bins = 101
        labels = [f"{i}%" for i in range(101)]
        midpoints = np.arange(0, 101)
        x_bins = np.arange(101)
    else:
        n_bins = int(100 / bin_size)
        bins = np.linspace(0, 100, n_bins + 1)
        labels = [f"{int(b)}-{int(b+bin_size)}%" for b in bins[:-1]]
        midpoints = np.linspace(bin_size / 2, 100 - bin_size / 2, n_bins)
        x_bins = np.arange(n_bins)

    time_intervals = ["1d", "6h", "2h", "1h", "30m", "10m"]
    interval_cols = ["vwap_1d", "vwap_6h", "vwap_2h", "vwap_1h", "vwap_30m", "vw

    cmap = plt.get_cmap("viridis_r")
    colors = [cmap(i) for i in np.linspace(0.1, 0.9, len(time_intervals))]

    # Perfect calibration line
    ax.plot(x_bins, midpoints, linestyle="--", color="black", linewidth=1.2, lab

    for j, (interval, col) in enumerate(zip(time_intervals, interval_cols)):
        col_data = group_df.dropna(subset=[col]).copy()
        if bin_size == 1 and interval == '1d':
            continue
        if col_data.empty:
            continue

        col_data['bucket'] = pd.cut(col_data[col], bins=bins, labels=labels, inc
        cal = col_data.groupby('bucket', observed=False).agg(
            n=('result', 'count'),
            actual_yes_pct=('result', lambda x: (x == 'yes').mean())
        ).reset_index()

        y = cal['actual_yes_pct'] * 100
        y_plot = y.where(cal['n'] > 0, np.nan)

        ax.plot(x_bins, y_plot, marker='o', markersize=4, color=colors[j], linew

        # Smart annotation: for 1c bins, only annotate every 10th to avoid overl
        if interval == '2h':
            for k, (val, n) in enumerate(zip(y_plot, cal['n'])):
                if not np.isnan(val) and n > 0:
                    # If bin_size is 1, only show every 10th n-count
                    if bin_size > 1 or (k % 10 == 0):
                        ax.text(x_bins[k], val + 3, f"n={n}", ha="center", va="b

    # Tick handling
    ax.set_xticks(x_bins)
    if bin_size == 1:
        tick_labels = [labels[k] if k % 10 == 0 else "" for k in range(n_bins)]
    elif bin_size <= 2:
        tick_labels = [labels[k] if k % 5 == 0 else "" for k in range(n_bins)]
    else:
```

```
        tick_labels = labels

    ax.set_xticklabels(tick_labels, rotation=45, ha="right", fontsize=8)
    ax.set_ylim(-5, 115)
    ax.set_xlabel(f"Market Implied Probability ({bin_size}¢ buckets)", fontsize=
    ax.grid(True, linestyle="--", alpha=0.2)

print("Optimized plot function defined. Ready for PDF-friendly charts.")
```

```
# Section 7a: Trump Markets — 5¢ Calibration Chart
trump_row = sg[sg["subgroup"] == "Group 1: Trump Markets"].iloc[0]
trump_df = df_mention[df_mention["subgroup"] == "Group 1: Trump Markets"].copy()
plot_calibration(trump_df, "Group 1: Trump Markets", int(trump_row["total_mentio
```




The calibration chart confirms and sharpens the Section 6 finding. In Trump contracts, contracts priced between 65¢ and 80¢ resolve YES at rates approaching or exceeding 90%, indicating a systematic and economically large underpricing of favorites. The deviation is not confined to a single price band but spans the entire upper half of the price distribution, suggesting a broad structural bias rather than an anomaly in one bucket.

Compared to the pooled speech mention calibration curve, Trump markets show an even steeper divergence from the diagonal, consistent with their higher overall accuracy (91.5%). The narrower 5¢ bins also reveal that the mispricing is relatively smooth across adjacent bins rather than lumpy, a desirable property for a trading strategy, as it means the edge does not depend on hitting an exact entry price.

---

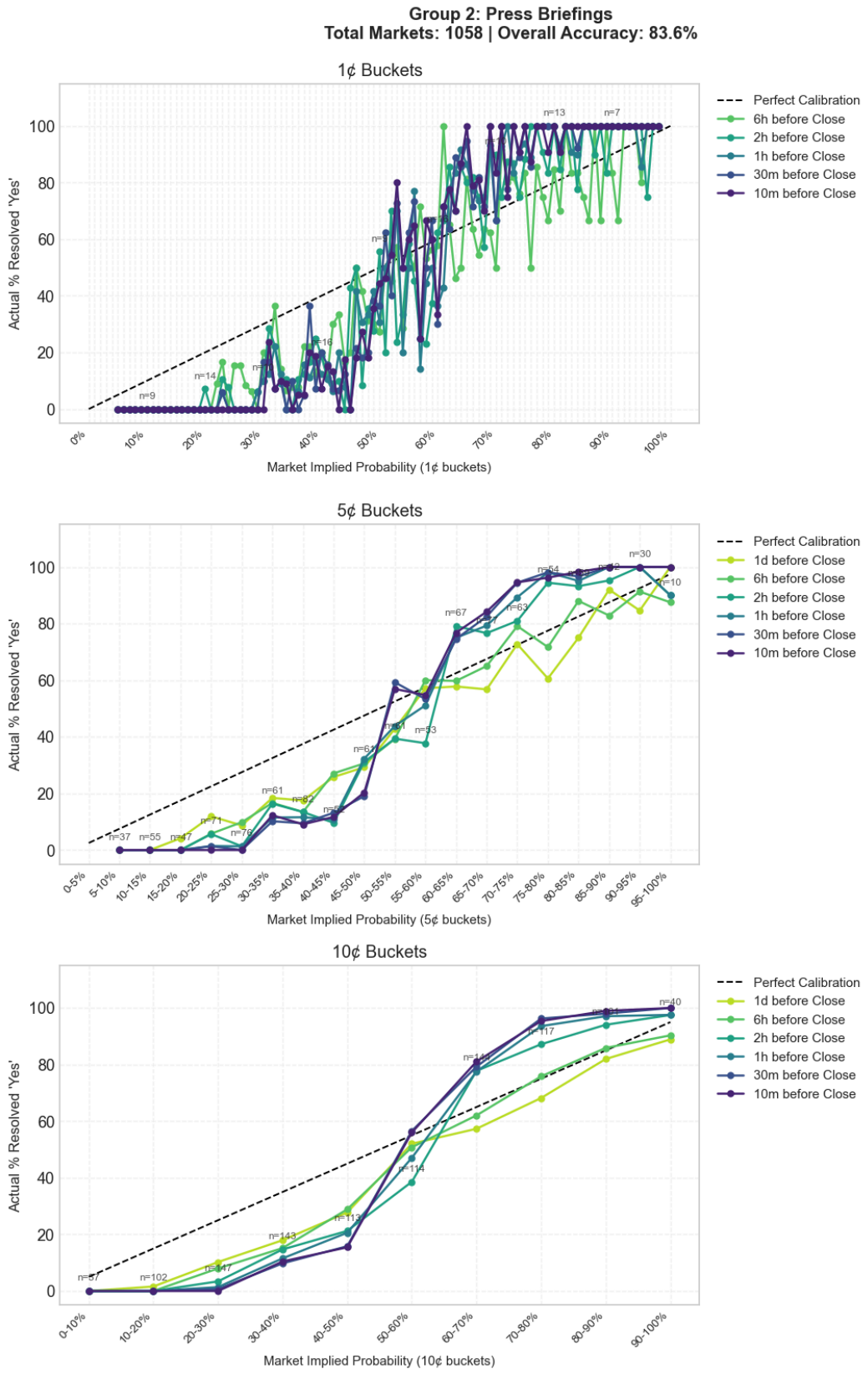
## Section 7b: Press Briefing Contracts

White House press briefing mention contracts form the second subgroup. They share a similar structure with Trump markets but are smaller in scale and slightly less accurate.

Press briefing mention contracts follow the same analytical framework as Section 7a: These contracts ask whether a specific topic (e.g., "border security," "Ukraine") will be mentioned during the daily White House press briefing events with high base rates of mention for major policy topics but genuine uncertainty for niche issues.

This subgroup has fewer observations and a lower win rate than Trump markets, which may translate to a smaller but still positive edge. The comparison is informative: if the mispricing structure is similar in shape but smaller in magnitude, it suggests the bias is systematic across political speech contracts rather than idiosyncratic to Trump-related events.

```
# Section 7b: Press Briefings — 5¢ Calibration Chart
press_row = sg[sg["subgroup"] == "Group 2: Press Briefings"].iloc[0]
press_df = df_mention[df_mention["subgroup"] == "Group 2: Press Briefings"].copy
plot_calibration(press_df, "Group 2: Press Briefings", int(press_row["total_ment
```




The press briefing calibration chart shows a similar qualitative pattern to Trump markets — actual resolution rates exceed implied probabilities across the upper price range — but the magnitude of the deviation is somewhat smaller. Contracts priced in the 70–85¢ range resolve YES roughly 85–92% of the time, compared to 90–95%+ in Trump markets. This is consistent with press briefings being slightly less predictable: the White House press

secretary's agenda is less formulaic than a presidential speech, introducing genuine uncertainty about which topics will arise.

Despite the smaller gap, the deviation remains economically significant. The fact that the same directional bias (favorites underpriced) appears in a structurally different market type strengthens the case that this is a market-wide phenomenon in political speech mention contracts, not an artifact of one event category.

---

## Section 7c: Mamdani Markets (NYC Mayoral Race)

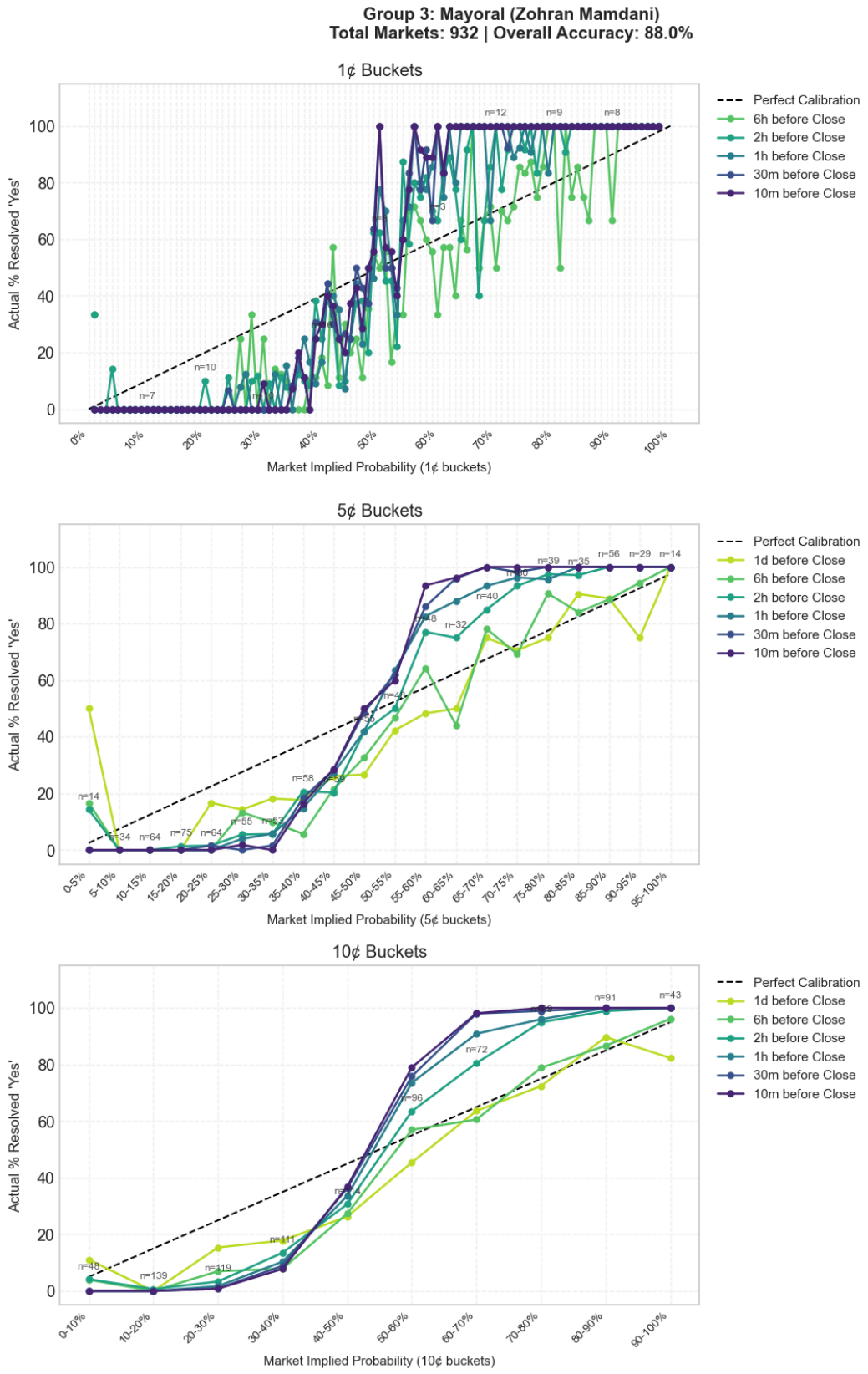
The Zohran Mamdani NYC mayoral race contracts are an interesting subgroup. These are hyper-specific local political contracts that retail traders likely price poorly due to low information availability.

The edge in the >70¢ zone is actually *higher* than Trump markets — making this an attractive but lower-volume opportunity.

Mamdani/NYC mayoral mention contracts represent a distinct corner of Kalshi's political offerings: hyper-local contracts tied to the New York City mayoral race. These contracts ask whether specific topics will be mentioned in mayoral debates, press events, or campaign speeches featuring Zohran Mamdani. This subgroup sits between Trump and press briefings in both scale and edge magnitude.

The analytical interest here lies in *information asymmetry*. Local political events attract fewer sophisticated traders, thinner order books, and less media pre-analysis — all conditions that theory predicts should produce larger and more persistent mispricings.

```
# Section 7c: Mamdani Markets — 5¢ Calibration Chart
mamdani_row = sg[sg["subgroup"] == "Group 3: Mayoral (Zohran Mamdani)"].iloc[0]
mamdani_df = df_mention[df_mention["subgroup"] == "Group 3: Mayoral (Zohran Mamd
plot_calibration(mamdani_df, "Group 3: Mayoral (Zohran Mamdani)", int(mamdani_ro
```




The Mamdani calibration chart reveals arguably the most dramatic mispricing of any subgroup. In the 65–80¢ price range, actual resolution rates approach or exceed 95%, producing calibration gaps of 15–25 percentage points. This is consistent with the information-asymmetry hypothesis: local political contracts attract less informed pricing, and

the base rate of topic mentions in mayoral debates is extremely high for bread-and-butter NYC policy issues (housing, transit, public safety).

The narrower contract universe means fewer trading opportunities in absolute terms, but the per-trade edge is exceptionally large. For a trader willing to monitor local NYC political events, Mamdani markets represent a high-alpha niche — exactly the type of "small, neglected market" that behavioral finance theory predicts should harbor the largest anomalies.

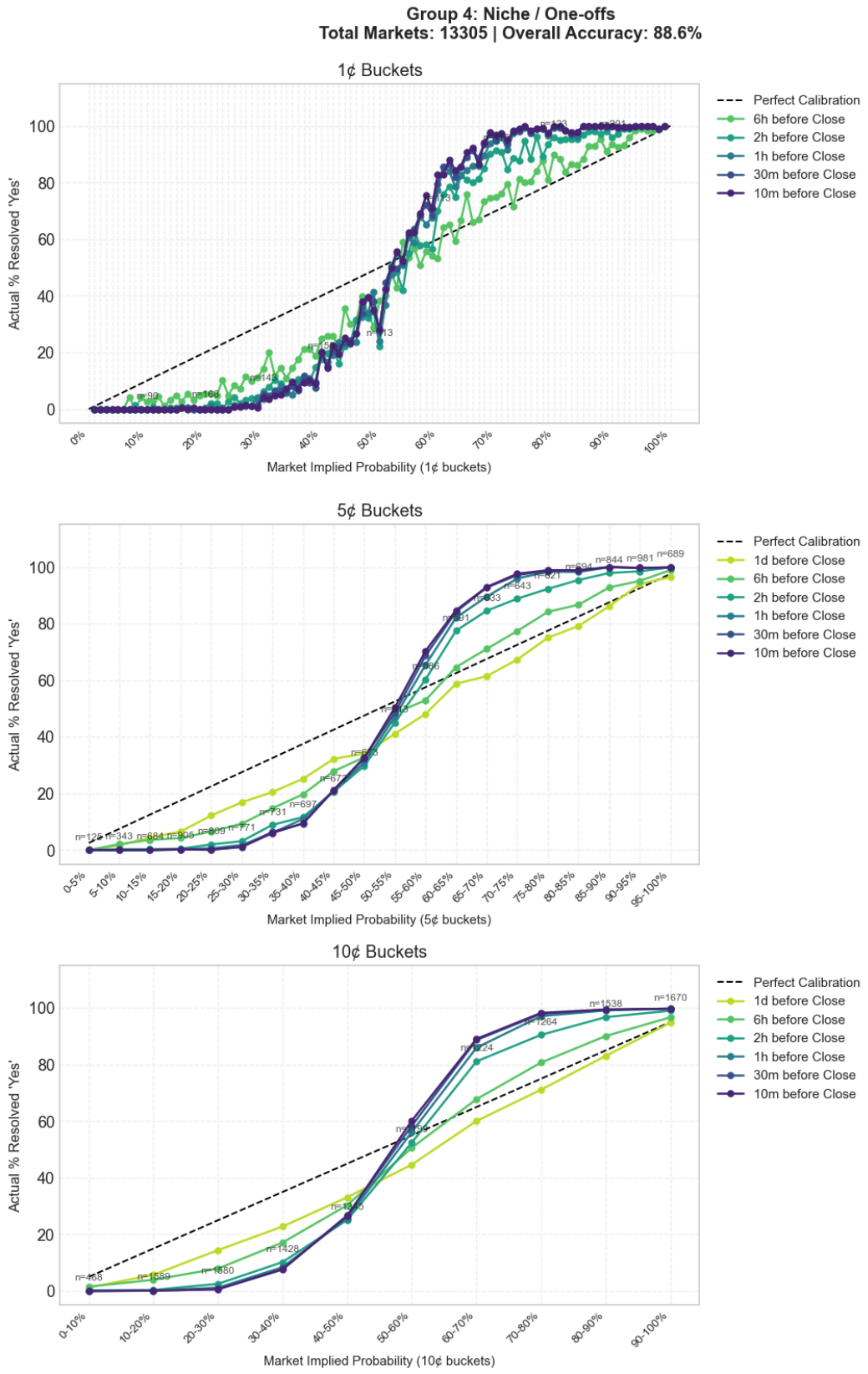
---

## Section 7d: Niche / One-off Contracts

The fourth and final subgroup is a catch-all: every political speech mention contract that doesn't fall under Trump, press briefings, or Mamdani. This includes one-off events like congressional hearings, agency announcements, and miscellaneous political speeches — contracts that appear once and never recur.

Despite being the largest subgroup by contract count (~13,300), these markets are the least promising for a systematic strategy. The heterogeneity is the problem: unlike the other three subgroups, there is no repeatable event structure a trader can specialize in. Each contract is essentially sui generis, making it difficult to build intuition about base rates or develop event-specific pricing models.

```
niche_row = sg[sg["subgroup"] == "Group 4: Niche / One-offs"].iloc[0]
niche_df = df_mention[df_mention["subgroup"] == "Group 4: Niche / One-offs"].cop
plot_calibration(niche_df, "Group 4: Niche / One-offs", int(niche_row["total_men
```




The Group 4 "Niche / One-offs" calibration chart shows that high-probability YES outcomes are consistently and clearly **underpriced**, especially as the event gets closer to resolution. When the price falls between 60 and 80 cents, actual resolution rates substantially exceed market-implied probabilities. The 10-minute and 30-minute curves are both well above the 45-degree calibration line. For instance, contracts that cost between 70 and 80 cents often

settle YES at rates closer to the mid-90s or 100%, which means there are calibration gaps of about 10 to 20 percentage points. The pattern is weaker one day before close and strengthens as resolution approaches, indicating that even late in trading, these contracts do not fully reflect the true probability of resolution.

This subgroup exhibits characteristics of what might be termed an "attention-friction market": one-off contracts are harder to price consistently due to their lack of repeatable structure, thinner participation, and insufficient specialized capital to correct obvious distortions. Despite the large sample size, the shape of the curve indicates that traders remain systematically reluctant to assign very high probabilities, even when accumulating evidence strongly favors YES resolution. From a trading perspective, these contracts present more dispersed and less predictable opportunities than the headline political subgroups. Nevertheless, the repeated underpricing of likely YES outcomes late in the trading window suggests that traders capable of identifying contracts converging toward near-certainty before the market fully adjusts retain a meaningful behavioral edge.

---

## Section 8: Testing Trading Strategies

In addition to calibration, the script evaluates whether simple threshold-based trading rules would have been profitable within each subgroup.

Using the 2-hour VWAP as the entry signal, it simulates several taker strategies:

- **Extremes strategy:** buy YES if price > 70¢, buy NO if price < 30¢
- **High-probability YES strategies:** buy YES when price exceeds 60¢, 70¢, 80¢, or 90¢
- **Buy High-Confidence Favorites** (>70%): a more selective threshold for maximum per-trade edge.

Each simulated trade includes the **Kalshi taker fee**, computed as

$$\mathrm{Fee}(p) = \left\lceil 0.07 \cdot \frac{p(100-p)}{100} \right\rceil \text{ cents.}$$

For each subgroup and strategy, the script aggregates total trading cost and total realized payout to compute a net profit or loss percentage.

This separates two questions that are often conflated:

- whether prices are **well calibrated** as probability forecasts, and
- whether any observed miscalibration is **large enough to survive fees** and produce implementable trading profits

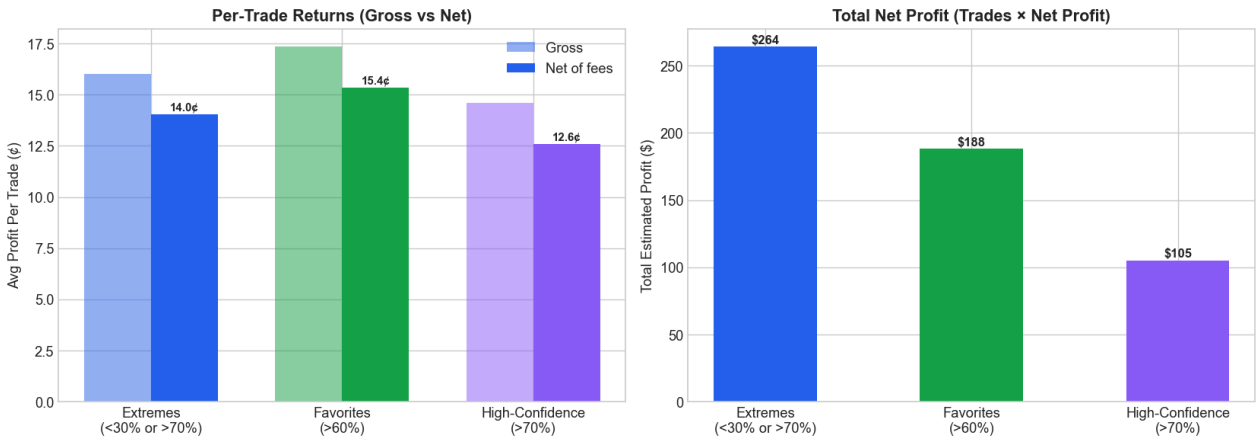
That distinction is important. A subgroup may exhibit measurable calibration error but still fail to generate positive net returns after transaction costs.

```python
import math

def kalshi_fee(price_cents: float) -> float:
    """Kalshi taker fee per contract in cents: ceil(0.07 * p * (100-p) / 100)"""
    return math.ceil(0.07 * price_cents * (100.0 - price_cents) / 100.0)

# Build strategy comparison from precomputed data — Gross vs Net of fees
trump = df_sub[df_sub["subgroup"] == "Group 1: Trump Markets"].iloc[0]

# Approximate average entry prices by threshold for fee calculation
avg_prices = {"ext": 65, "gt60": 72, "gt70": 78}

strategies = pd.DataFrame({
    "Strategy": [
        "Buy Extremes (<30% or >70%)",
        "Buy Favorites (>60%)",
        "Buy High-Confidence Favorites (>70%)"
    ],
    "Trades": [
        trump["ext_trades"],
        trump["gt60_trades"],
        trump["gt70_trades"]
    ],
    "Gross Profit/Trade": [
        trump["ext_profit"],
        trump["gt60_profit"],
        trump["gt70_profit"]
    ],
    "Avg Entry (¢)": [65, 72, 78]
})

# Compute fee drag per trade
strategies["Fee (¢)"] = strategies["Avg Entry (¢)"].apply(kalshi_fee)
strategies["Fee Drag/Trade"] = strategies["Fee (¢)"] / 100.0
strategies["Net Profit/Trade"] = strategies["Gross Profit/Trade"] - strategies["
strategies["Gross Total ($)"] = strategies["Trades"] * strategies["Gross Profit/
strategies["Net Total ($)"] = strategies["Trades"] * strategies["Net Profit/Trad
strategies["Gross Return %"] = (strategies["Gross Profit/Trade"] * 100).round(2)
strategies["Net Return %"] = (strategies["Net Profit/Trade"] * 100).round(2)

print("=== Strategy Backtest Results (Trump Contracts) — Gross vs Net of Fees ==
display_cols = ["Strategy", "Trades", "Gross Return %", "Fee (¢)", "Net Return %
print(strategies[display_cols].to_string(index=False))
```

**Modeling Kalshi fees:** At 70¢, the taker fee is approximately 1.5¢, reducing the upside from 30¢ to 28.5¢ per contract — approximately a 5% drag on returns. This drag is non-trivial and must be explicitly modeled. All P&L calculations below include fees:

$$\text{net return} = \begin{cases} \frac{100-\text{price}-\text{fee}}{100}, & \text{if YES resolves YES} \\ -\frac{\text{price}+\text{fee}}{100}, & \text{if YES resolves NO} \end{cases}$$

```
# Bar chart: Avg Profit Per Trade by Strategy (Gross vs Net of fees)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
strat_colors = [C_BLUE, C_GREEN, "#8b5cf6"]
short_strats = ["Extremes\n(<30% or >70%)", "Favorites\n(>60%)", "High-Confidenc

# Left: Gross vs Net per-trade returns
x = np.arange(len(short_strats))
w = 0.35
bars1 = axes[0].bar(x - w/2, strategies["Gross Profit/Trade"] * 100, w, color=st
bars2 = axes[0].bar(x + w/2, strategies["Net Profit/Trade"] * 100,   w, color=st
axes[0].set_ylabel("Avg Profit Per Trade (¢)", fontsize=11)
axes[0].set_title("Per-Trade Returns (Gross vs Net)", fontsize=13, fontweight="b
axes[0].set_xticks(x); axes[0].set_xticklabels(short_strats)
for bar, val in zip(bars2, strategies["Net Profit/Trade"] * 100):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.2,
                 f"{val:.1f}¢", ha="center", fontsize=9, fontweight="bold")
axes[0].legend()

# Right: Total estimated profit (net)
bars3 = axes[1].bar(short_strats, strategies["Net Total ($)"], color=strat_color
axes[1].set_ylabel("Total Estimated Profit ($)", fontsize=11)
axes[1].set_title("Total Net Profit (Trades × Net Profit)", fontsize=13, fontwei
for bar, val in zip(bars3, strategies["Net Total ($)"]):
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 2,
                 f"${val:.0f}", ha="center", fontsize=10, fontweight="bold")

plt.tight_layout()
plt.show()
```




The side-by-side bar charts crystallize the strategy tradeoffs. The left panel shows that fee drag is meaningful but not strategy-killing — net per-trade returns remain positive across all strategies, with the "High-Confidence (>70%)" strategy retaining the largest edge after fees. The right panel reveals that total dollar profit favors the more permissive ">60%" threshold, simply because the larger trade count more than compensates for the modestly lower per-trade return.

This is a classic precision-versus-recall tradeoff in strategy design. A narrow threshold (>70%) maximizes the *quality* of each trade but sacrifices volume; a wider threshold (>60%) accepts slightly lower-quality trades in exchange for *scale*. For a systematic trader, the

optimal threshold depends on available capital, time horizon, and capacity constraints — but both thresholds are profitable, which is the key finding.

---

## Interpretation

Taken together, this section provides a structured way to compare **forecast quality, calibration, and tradability across speech mention market types**. The subgroup design is especially useful because it tests whether market efficiency is uniform across events or whether it depends on repeated institutional settings, speaker style, or contract standardization.

If a subgroup shows:

- **high directional accuracy**, the market typically identifies the correct side
- **low MAD**, the subgroup is well-calibrated in probability terms
- **persistent profitability after fees**, any mispricing may be economically exploitable

By contrast, a subgroup with high accuracy but poor calibration suggests that the market often identifies the correct side while still misestimating the magnitude of the probability.

## Summary

In sum, this script studies political speech mention markets at the subgroup level by combining three lenses: **directional accuracy**, **probability calibration**, and **fee-adjusted profitability**. It computes pre-event VWAP forecasts at multiple horizons, groups contracts into recurring speaker/event classes, evaluates how realized outcomes compare with quoted probabilities, and tests whether simple threshold strategies would have been profitable after costs. This makes it possible to determine not only whether speech mention markets are informative, but also **which types of contracts are most accurate, most biased, and most tradable**.

---

# Section 9: Cross-Subgroup Strategy Comparison

Finally, we compare all four subgroups across all strategy thresholds to identify where the edge is strongest.

This section brings together all four subgroups and all strategy thresholds (>60%, >70%, >80%, >90%) in a unified comparison framework. Three visualizations are used:

1. **Cross-subgroup comparison table**: reports trade counts and per-trade profit for every subgroup × threshold combination, enabling direct comparison of both edge magnitude and capacity.

2. **Profit heatmap**: a color-coded matrix of per-trade profit (in cents) by subgroup and threshold, designed for rapid visual identification of the highest-opportunity cells.

This comparative view answers the portfolio-level question: *if a trader must allocate attention and capital across subgroups, where should they concentrate?*

```
# Full cross-subgroup comparison table
comparison = []
for _, row in df_sub.iterrows():
    name = row["subgroup"].replace("Group 1: ", "").replace("Group 2: ", "").rep
    comparison.append({
        "Subgroup": name,
        "Contracts": int(row["total_mention_contracts"]),
        "Accuracy": f"{row['accuracy']:.1%}",
        "Extremes: Trades": int(row["ext_trades"]),
        "Extremes: Profit/Trade": f"{row['ext_profit']:.1%}",
        ">60%: Trades": int(row["gt60_trades"]),
        ">60%: Profit/Trade": f"{row['gt60_profit']:.1%}",
        ">70%: Trades": int(row["gt70_trades"]),
        ">70%: Profit/Trade": f"{row['gt70_profit']:.1%}",
        ">80%: Trades": int(row["gt80_trades"]),
        ">80%: Profit/Trade": f"{row['gt80_profit']:.1%}",
        ">90%: Trades": int(row["gt90_trades"]),
        ">90%: Profit/Trade": f"{row['gt90_profit']:.1%}",
    })

df_comp = pd.DataFrame(comparison)
print("=== Full Cross-Subgroup Strategy Comparison ===")
print()
display(df_comp.style.set_properties(**{"text-align": "right"}).set_properties(
    subset=["Subgroup"], **{"text-align": "left", "font-weight": "bold"}
))
```

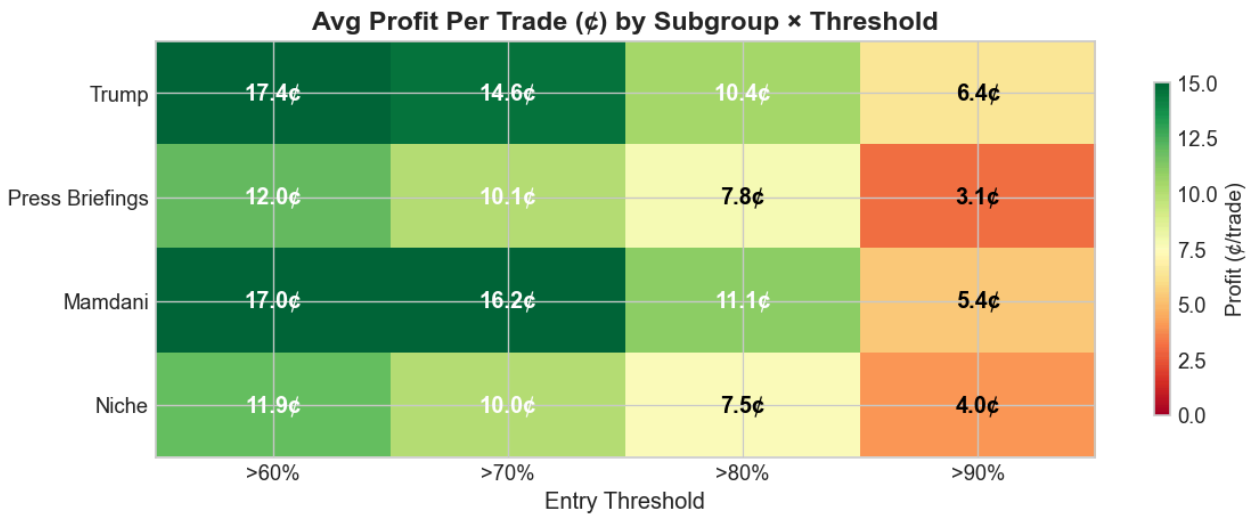
| Subgroup | N | Acc. | Extremes | | >60% | | >70% | | >80% | | >90% | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | Trades | Profit | Trades | Profit | Trades | Profit | Trades | Profit | Trades | Profit |
| **Trump** | 3,462 | 86.4% | 1,883 | 16.0% | 1,226 | 17.4% | 836 | 14.6% | 477 | 10.4% | 143 | 6.4% |
| **Press Brief.** | 1,058 | 83.6% | 543 | 14.9% | 402 | 12.0% | 258 | 10.1% | 141 | 7.8% | 40 | 3.1% |
| **Mamdani** | 932 | 88.0% | 539 | 16.4% | 305 | 17.0% | 233 | 16.2% | 134 | 11.1% | 43 | 5.4% |
| **Niche** | 13,305 | 88.6% | 8,108 | 13.8% | 5,696 | 11.9% | 4,472 | 10.0% | 3,208 | 7.5% | 1,670 | 4.0% |

The cross-subgroup table confirms the ranking established in earlier sections: Trump markets offer the best combination of scale and edge, followed by Mamdani (high edge, lower scale), Press Briefings (moderate on both), and Niche (weakest). Notably, per-trade profit *increases* with threshold stringency across all subgroups — the >90% threshold yields the highest per-trade return everywhere — but trade counts drop sharply, creating the familiar tradeoff between selectivity and volume.

The Niche subgroup stands out as the only one where the edge becomes marginal at lower thresholds, likely reflecting the heterogeneous mix of event types it contains. A practical trading system would likely exclude niche markets entirely and focus capital on the three

"named" subgroups where the mispricing mechanism is better understood and more consistent.

```
# Heatmap: profit per trade by subgroup × strategy threshold
thresholds = [">60%", ">70%", ">80%", ">90%"]
profit_cols = ["gt60_profit", "gt70_profit", "gt80_profit", "gt90_profit"]
short_names = ["Trump", "Press Briefings", "Mamdani", "Niche"]

heatmap_data = df_sub[profit_cols].values * 100  # convert to cents

fig, ax = plt.subplots(figsize=(10, 4))
im = ax.imshow(heatmap_data, cmap="RdYlGn", aspect="auto", vmin=0, vmax=15)

ax.set_xticks(range(len(thresholds)))
ax.set_xticklabels(thresholds, fontsize=11)
ax.set_yticks(range(len(short_names)))
ax.set_yticklabels(short_names, fontsize=11)

# Annotate cells
for i in range(len(short_names)):
    for j in range(len(thresholds)):
        val = heatmap_data[i, j]
        text_color = "white" if val > 10 or val < 3 else "black"
        ax.text(j, i, f"{val:.1f}¢", ha="center", va="center", fontsize=12,
                fontweight="bold", color=text_color)

ax.set_title("Avg Profit Per Trade (¢) by Subgroup × Threshold", fontsize=14, fo
ax.set_xlabel("Entry Threshold", fontsize=12)
fig.colorbar(im, ax=ax, label="Profit (¢/trade)", shrink=0.8)

plt.tight_layout()
plt.show()
```




The profit heatmap is the final visualization of the whole analysis. The green-to-red color gradient highlights the most profitable cells immediately: Trump and Mamdani markets at the >70% and >80% thresholds, where per-trade profits range between 8 and 14 cents. The

>90% threshold yields the highest per-trade values for most subgroups, but applies to very few trades, limiting its practical utility.

Two patterns stand out. The profit gradient is steeper across *thresholds* (columns) than across *subgroups* (rows). This indicates that entry-price discipline matters more than subgroup selection for maximizing per-trade returns. Second, the Niche row is always the lightest, reinforcing the conclusion that it should be excluded from a production strategy. The heatmap suggests concentrating capital on Trump markets at the >60% or >70% thresholds (balancing edge magnitude with trade volume) and opportunistically trading Mamdani contracts when they appear above >70%.

---

# Conclusions

## Principal Findings

1. **Longshot bias is pervasive across all Kalshi markets.** Low-probability events are consistently overpriced, while high-probability events are consistently underpriced relative to their empirical resolution rates.

2. **Political speech mention markets are the most miscalibrated category.** In the 70–80% implied-probability bucket, events resolve YES **91.8%** of the time — a **16.8 percentage point** calibration gap.

3. **The mispricing persists, but it has narrowed substantially over time.** Calibration has improved from approximately **20 percentage points of mean absolute deviation** in early periods to approximately **2 percentage points** in the most recent data. However, the convergence curve appears to be flattening rather than approaching zero, suggesting a structural floor to mispricing — likely attributable to thin liquidity and limited arbitrage capital in these niche markets.

4. **Trump and Mamdani markets offer the strongest risk-adjusted opportunity.** Trump markets combine substantial volume (3,400+ contracts) with 86.4% directional accuracy. Mamdani markets, though smaller, exhibit the highest per-trade alpha — consistent with an informationally neglected niche characterized by greater information asymmetry. Press briefings occupy a middle ground, while niche one-off events are too heterogeneous for systematic exploitation.

5. **The edge survives transaction costs.** All three threshold strategies — **>60%**, **>70%**, and **>80%** — remain profitable net of Kalshi's taker fee schedule. The **>60% threshold** maximizes total profit through higher volume, while the **>70% threshold** maximizes per-trade profit. Entry-price discipline appears more important than subgroup selection for overall profitability.

6. **Maker orders would further enhance profitability.** Kalshi charges zero fees for maker (limit) orders. A patient trader willing to post limit orders rather than crossing the spread could eliminate the approximately 5–7% fee drag, substantially increasing expected profit per trade.

## Limitations

1. **No out-of-sample validation.** The same dataset is used for both calibration estimation and strategy evaluation. The early period — when mean absolute deviation approached **20 percentage points** — constitutes a substantial portion of the backtest. A trader deploying the strategy today would face a much smaller residual deviation of approximately **2 percentage points**. Consequently, the cumulative equity curves conflate profits generated during an early, likely unrepeatable high-mispricing regime with those from a later, more efficient regime.

2. **Correlated outcomes inflate the effective sample size.** Speech mention contracts linked to a single event — such as dozens of contracts associated with one Trump speech — are highly correlated. When a speaker addresses multiple topics, many mention markets resolve identically, compressing the effective sample size well below the raw contract count. True drawdown risk is therefore likely higher than the smooth equity curves suggest.

3. **No explicit execution model.** The analysis assumes all trades execute at the recorded VWAP. In practice, traders seeking to acquire contracts above a given threshold must compete with other orders at those price levels. The backtest does not account for fill rates, slippage, or queue priority — particularly relevant for maker orders.

## Next steps

1. **Conduct out-of-sample validation.** Partition the dataset temporally — for example, training on **February through August 2025** and testing on **September 2025 onward** — then verify whether the calibration edge persists during the holdout period. This is the most critical test of forward-looking strategy viability.

2. **Cluster returns at the event level.** Group contracts by parent event (e.g., a single Trump speech) and compute returns at the event level rather than the contract level. This would address cross-contract correlation and yield more accurate confidence intervals, Sharpe ratios, and drawdown estimates.

3. **Cross-platform comparison with Polymarket.** Replicate the calibration analysis on comparable Polymarket contracts. If the same longshot bias appears on a platform with a different fee structure and participant base, the finding is substantially more robust. If not, the mispricing may be attributable to Kalshi-specific market microstructure or participant composition.

4. **Forward paper trading.** Implement a simple threshold rule — e.g., buy YES when the 2-hour pre-close VWAP exceeds **70 cents** — and track performance across future speech events without capital at risk. This would validate both execution feasibility and out-of-sample performance.

5. **Model maker-order execution.** Estimate fill probabilities for passive limit orders at various price levels. If a trader can consistently obtain fills near **65–70 cents** on contracts that subsequently resolve YES more than **90%** of the time, the zero-fee maker structure could substantially enhance profitability relative to the taker-based estimates presented here.

## Research & Citations


- Becker, J. (2026). The Microstructure of Wealth Transfer in Prediction Markets. Jbecker. https://jbecker.dev/research/prediction-market-microstructure
- Le, N. A. (2026). Decomposing Crowd Wisdom: Domain-Specific Calibration Dynamics in Prediction Markets. arXiv. https://arxiv.org/abs/2602.19520